# South Valley National University

# Course Book: Knowledge Discovery

# LEVEL4 Computer Science

# Code: CS464

# Course Objectives

By the end of this course, students will be able to:

1- Understand the fundamental concepts of Knowledge Discovery in Databases (KDD) and its relationship with Data Mining and Machine Learning.
2- Explain the stages of the KDD process, including data selection, preprocessing, transformation, data mining, and knowledge evaluation.
3- Apply various data mining techniques such as classification, clustering, association rule mining, and anomaly detection to extract useful patterns from data.
4- Analyze and interpret discovered patterns to support decision-making and problem-solving in real-world applications.
5- Use appropriate tools and software (e.g., WEKA, Python libraries, or SQL-based tools) to perform data exploration and pattern discovery.
6- Evaluate the performance and validity of discovered knowledge using metrics such as accuracy, precision, recall, and lift.
7- Integrate ethical and privacy considerations into the knowledge discovery process to ensure responsible use of data.
8- Develop small-scale projects that demonstrate practical application of KDD concepts in domains such as business, healthcare, or scientific research.

# Content

**Chapter 1: Introduction to Knowledge Discovery** ..... 4
1.1 What is Data Mining ..... 4
1.2 Knowledge Discovery (KDD) Process ..... 5
1.3 Applications of Data Mining ..... 7
**Chapter 2: Data Understanding and Data Preparation** ..... 10
2.1 Data Cleaning ..... 11
2.2 Data Integration ..... 13
2.3 Data Reduction ..... 15
2.4 Data Transformation ..... 17
**Chapter 3: Data Warehousing and Online Analytical Processing (OLAP)** ..... 22
3.1 Data Warehouse Architecture ..... 23
3.2 Enterprise Warehouse, Data Mart, and Virtual Warehouse ..... 25
3.3 OLAP Operations: Roll-up, Drill-down, Slice, Dice ..... 27
3.4 Data Warehouse Design and Implementation ..... 30
**Chapter 4: Data Cube and Multidimensional Data Model** ..... 35
4.1 Data Cube Concept ..... 36
4.2 Dimensions and Facts ..... 38
4.3 Multidimensional Schemas: Star, Snowflake, Galaxy ..... 41
4.4 Concept Hierarchies and Measures ..... 44
**Chapter 5: Pattern Discovery and Association Analysis** ..... 50
5.1 Association Rule Mining ..... 51
5.2 Apriori Algorithm ..... 54
5.3 FP-Growth Method ..... 58
5.4 Pattern Evaluation and Correlation Analysis ..... 63

# Chapter 1

# Introduction

## 1.1 what is data mining

The explosive growth of data, ranging from terabytes to petabytes, has completely transformed the modern world into a data-driven and computerized society. The rapid advancement of technology, automated collection tools, database systems, and the Web has made data generation and storage easier than ever before. Major sources of data include business activities such as e-commerce and transactions, scientific domains such as bioinformatics and remote sensing, and social interactions through platforms like news outlets, digital cameras, and YouTube.

Despite this abundance, organizations often face a critical problem: they are drowning in data but starving for knowledge. This imbalance between the volume of data and the ability to extract meaningful insights gave rise to the field of **Data Mining**, which provides automated analysis techniques for massive data sets.

The term **Data Mining**, also known as **Knowledge Discovery from Data (KDD)**, refers to the process of extracting interesting, non-trivial, implicit, previously unknown, and potentially useful patterns or knowledge from large volumes of data. It is sometimes referred to as knowledge extraction, data/pattern analysis, data archeology, information harvesting, or business intelligence. However, not every database operation qualifies as data mining. Simple search queries or expert systems only retrieve what is already known, while data mining focuses on discovering implicit relationships and hidden knowledge that were not evident before.

For instance, a company may discover through data mining that customers who purchase baby products tend to buy beer on Fridays — a relationship that would not appear in

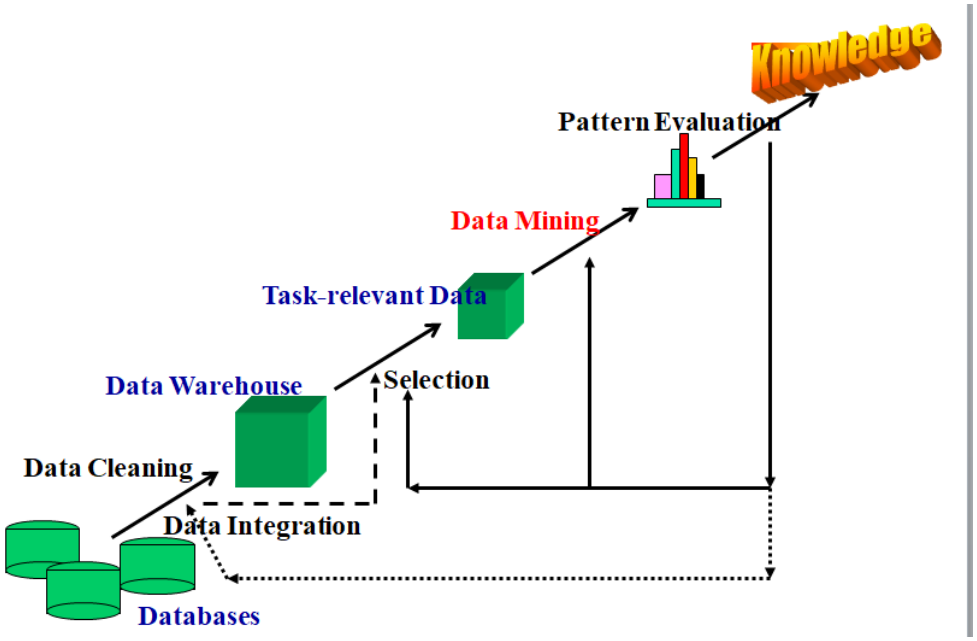
simple queries. Thus, data mining is not a retrieval process but a **discovery-oriented process** aimed at finding previously unknown insights that can guide decisions.

Data mining plays an essential role in the broader **Knowledge Discovery in Databases**

## 1.2 Knowledge Discovery (KDD) Process

**(KDD)** process, which consists of several stages that transform raw data into actionable knowledge. The process typically includes:

1. **Data Cleaning** – removing noise and inconsistent information.
2. **Data Integration** – combining data from multiple sources into a coherent dataset.
3. **Data Selection** – identifying relevant subsets of data for analysis.
4. **Data Transformation** – converting data into appropriate formats for mining.
5. **Data Mining** – applying algorithms to extract patterns.
6. **Pattern Evaluation** – assessing which patterns are truly interesting and valid.
7. **Knowledge Presentation** – visualizing or reporting the findings in an understandable form.




***[Figure1.1: Diagram showing the complete KDD process from raw data to knowledge]***

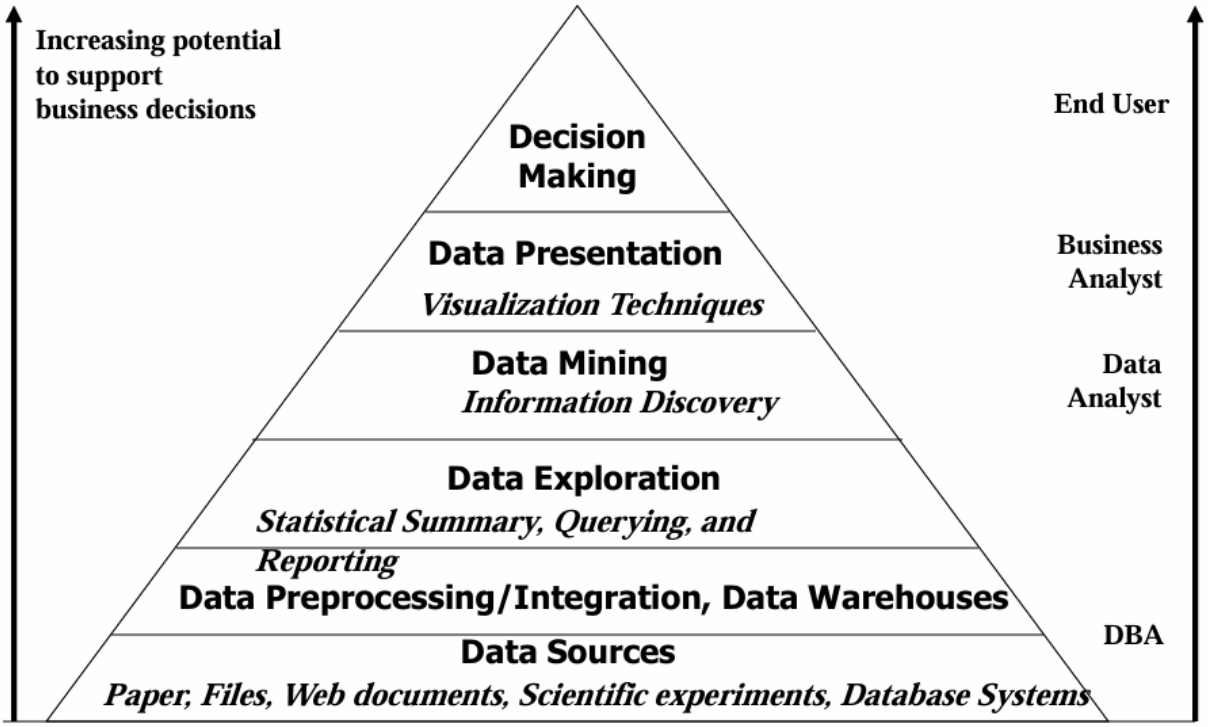
Each of these stages ensures that the resulting knowledge is accurate, useful, and relevant. Among them, **data mining** is the core analytical step that employs algorithms from statistics, artificial intelligence, and machine learning to extract patterns and models from large datasets.

A practical example of this process can be seen in **Web Mining**, a subfield of data mining that focuses on web data. It typically involves data cleaning, integration, warehousing, cube construction, selection, mining, and presentation. Web mining can be divided into three main areas:

- **Web Content Mining** – extracting information from page content.
- **Web Structure Mining** – analyzing link structures between pages.
- **Web Usage Mining** – studying user navigation behavior through log analysis.

## 1.3 Data Mining in Business Intelligence

In the context of **Business Intelligence (BI)**, data mining provides a foundation for informed decision-making. It supports a layered analytical process that begins with data sources (such as files, documents, and databases), followed by data preprocessing and integration into warehouses, exploration through queries and statistical summaries, mining for hidden patterns, and finally visualization for business decision-makers. BI systems rely on data mining to transition from simply describing what happened to predicting what will happen — a crucial element for strategic success.

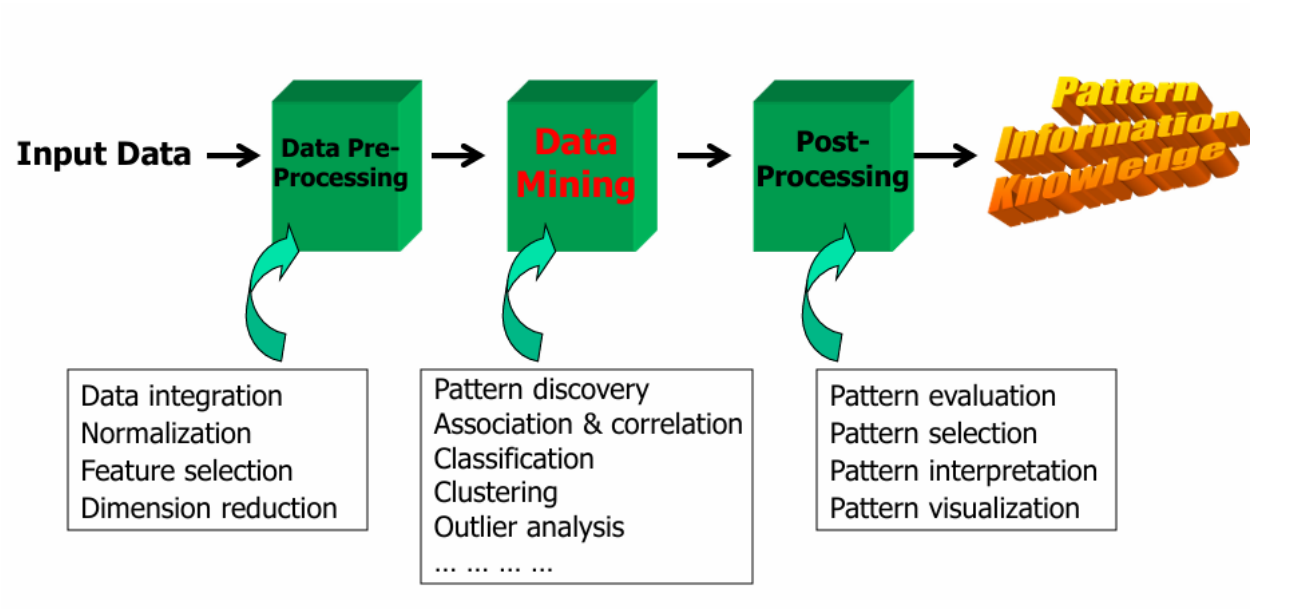



***[Figure1.2: Business Intelligence architecture with data mining as a core layer]***

## 1.4 KDD Process: A Typical View from ML and Statistics

From a **machine learning and statistical** perspective, the KDD process emphasizes model building and pattern recognition. Input data undergoes preprocessing steps such as normalization, feature selection, and dimensionality reduction before being mined. The post-processing stage then evaluates and visualizes the discovered patterns to ensure they are meaningful.

Data mining can be viewed from multiple dimensions. In terms of data, it deals with structured databases, warehouses, and streams; in terms of functions, it includes association, classification, clustering, and trend analysis; in terms of techniques, it incorporates OLAP, statistics, pattern recognition, and visualization; and in terms of applications, it covers diverse fields such as retail, telecommunications, banking, fraud detection, bioinformatics, and text or web mining.




Data mining can operate on many kinds of data: relational databases, data warehouses, transactional databases, data streams, time-series data, spatial and multimedia data, graphs, social networks, text databases, and the World Wide Web. This diversity highlights its flexibility and wide applicability.
The functions of data mining are equally diverse and include:

1. **Generalization** – summarizing and describing data characteristics.
2. **Association and Correlation Analysis** – discovering relationships among items, such as frequent co-purchases.
3. **Classification** – building predictive models using labeled examples.
4. **Cluster Analysis** – grouping data into meaningful clusters without predefined labels.
5. **Outlier Analysis** – detecting anomalies that differ from normal patterns.
6. **Sequential Pattern and Trend Analysis** – identifying time-based or evolving patterns.

Each function serves specific analytical goals. Association rules reveal co-occurrence relationships useful in marketing, while classification and clustering organize data for prediction or understanding. Outlier detection is essential for fraud detection or quality

control, and sequential analysis captures patterns over time, such as customer purchasing sequences or seasonal trends.

Not all discovered patterns are meaningful or useful. The **evaluation of knowledge** is a crucial step to ensure that only relevant, novel, and accurate patterns are retained. Evaluation measures include coverage, novelty, accuracy, and timeliness. Effective evaluation ensures that mined knowledge contributes to practical decision-making rather than overwhelming users with redundant information.

Data mining is inherently **interdisciplinary**, combining the strengths of several fields including machine learning, statistics, pattern recognition, database technology, high-performance computing, and data visualization.

The need for this confluence of disciplines arises from the growing complexity of modern data. Today's datasets are massive, high-dimensional, dynamic, and heterogeneous, encompassing everything from sensor readings to multimedia and web data. Handling such complexity requires methods that are both scalable and adaptive.

Finally, data mining can be summarized as the **process of discovering interesting, non-trivial, and useful patterns** from massive datasets. It represents a natural evolution of database technology, driven by the demand for better tools to extract actionable insights. The KDD process integrates data cleaning, integration, selection, transformation, mining, evaluation, and presentation, enabling the transformation of raw data into valuable knowledge applicable across business, science, and society.

## 1.5 Data mining: confluence of multiple disciplines

As a discipline that studies efficient and effective methods for uncovering patterns and knowledge from various kinds of massive data sets for many applications, data mining naturally serves a confluence of multiple disciplines including machine learning, statistics, pattern recognition, natural language processing, database technology, visualization and human computer interaction (HCI), algorithms, high-performance

computing, social sciences, and many application domains (Fig. 1.4). The interdisciplinary nature of data mining research and development contributes significantly to the success of data mining and its extensive applications. On the other hand, data mining is not only nurtured from the knowledge and development of these disciplines, the dedicated research, development, and applications of data mining on various kinds of big data may have substantially impacted the development of these disciplines in recent years as well. In this section, we discuss several disciplines that strongly impact and actively interact with the research, development, and applications of data mining.

### 1.5.1 Statistics and data mining

**Figure 1.3: Statistics and data mining**

Statistics studies the collection, analysis, interpretation or explanation, and presentation of data. Data mining has an inherent connection with statistics. A statistical model is a set of mathematical functions that describe the behavior of the objects in a target class in terms of random variables and their associated probability distributions. Statistical models are widely used to model data and data classes. For example, in data mining tasks such as data characterization and classification, statistical models of target classes can be built. In other words, such statistical models can be the outcome of a data mining

task. Alternatively, data mining tasks can be built on top of statistical models. For example, we can use statistics to model noise and missing data values. Then, when mining patterns in a large data set, the data mining process can use the model to help identify and handle noisy or missing values in the data. Statistics research develops tools for prediction and forecasting using data and statistical models. Statistical methods can be used to summarize or describe a collection of data. Basic statistical descriptions of data are introduced in Chapter 2. Statistics is useful for mining various patterns from data and for understanding the underlying mechanisms generating and affecting the patterns. Inferential statistics (or predictive statistics) models data in a way that accounts for randomness and uncertainty in the observations and is used to draw inferences about the process or population under investigation. Statistical methods can also be used to verify data mining results. For example, after a classification or prediction model is mined, the model should be verified by statistical hypothesis testing. A statistical hypothesis test (sometimes called confirmatory data analysis) makes statistical decisions using experimental data. A result is called statistically significant if it is unlikely to have occurred by chance. If the classification or prediction model holds, then the descriptive statistics of the model increases the soundness of the model. Applying statistical methods in data mining is far from trivial. Often, a serious challenge is how to scale up a statistical method over a large data set. Many statistical methods have high complexity in computation. When such methods are applied on large data sets that are also distributed on multiple logical or physical sites, algorithms should be carefully designed and tuned to reduce the computational cost. This challenge becomes even tougher for online applications, such as online query suggestions in search engines, where data mining is required to continuously handle fast, real-time data streams. Data mining research has developed many scalable and effective solutions for the analysis of massive data sets and data streams. Moreover, different kinds of data sets and different applications may require rather different analysis methods. Effective solutions have been

proposed and tested, which leads to many new, scalable data mining-based statistical analysis methods.

## Exercises

1.1. What is data mining? In your answer, address the following:

a. Is it a simple transformation or application of technology developed from databases, statistics, machine learning, and pattern recognition?

b. Someone believes that data mining is an inevitable result of the evolution of information technology. If you are a database researcher, show data mining is resulted from a nature evolution of database technology. What about if you are a machine learner researcher, or a statistician?

c. Describe the steps involved in data mining when viewed as a process of knowledge discovery.

1.2. Define each of the following data mining functionalities: association and correlation analysis, classification, regression, clustering, deep learning, and outlier analysis. Give examples of each data mining functionality, using a real-life database that you are familiar with.

1.3. Present an example where data mining is crucial to the success of a business. What data mining functionalities does this business need (e.g., think of the kinds of patterns that could be mined)? Can such patterns be generated alternatively by data query processing or simple statistical analysis?

1.4. Explain the difference and similarity between correlation analysis and classification, between classification and clustering, and between classification and regression

# Chapter 2

# Getting To Know Your Data

It's tempting to jump straight into mining, but first, we need to get the data ready. This involves having a closer look at attributes and data values. Real-world data are typically noisy, enormous in volume (often several gigabytes or more), and may originate from a hodgepodge of heterogeneous sources. This chapter is about getting familiar with your data. Knowledge about your data is useful for data preprocessing, the first major task of the data mining process. You will want to know the following: What are the types of attributes or fields that make up your data? What kind of values does each attribute have? Which attributes are discrete, and which are continuous-valued? What do the data look like? How are the values distributed? Are there ways we can visualize the data to get a better sense of it all? Can we spot any outliers? Can we measure the similarity of some data objects with respect to others? Gaining such insight into the data will help with the subsequent analysis.

The field of data visualization provides many additional techniques for viewing data through graphical means. These can help identify relations, trends, and biases "hidden" in unstructured data sets. Techniques may be as simple as scatter-plot matrices

Two attributes are mapped onto a 2-D grid) to more sophisticated methods such as tree maps (where a hierarchical partitioning of the screen is displayed based on the attribute values).

Finally, we may want to examine how similar (or dissimilar) data objects are. For example, suppose we have a database where the data objects are patients, described by their symptoms. We may want to find the similarity or dissimilarity between individual patients. Such information can allow us to find clusters of like patients within the data set. The similarity/dissimilarity between objects may also be used to detect outliers in the data or to perform nearest-neighbor classification. There are many measures for

assessing similarity and dissimilarity. In general, such measures are referred to as proximity measures. Think of the proximity of two objects as a function of the distance between their attribute values, although proximity can also be calculated based on probabilities rather than actual distance.

In summary, by the end of this chapter, you will know the different attribute types and basic statistical measures to describe the central tendency and dispersion (spread) of attribute data. You will also know techniques to visualize attribute distributions and how to compute the similarity or dissimilarity between objects.

## 2.1 Data Objects and Attribute Types

Data sets are made up of data objects. A data object represents an entity—in a sales database, the objects may be customers, store items, and sales; in a medical database, the objects may be patients; in a university database, the objects may be students, professors, and courses. Data objects are typically described by attributes. Data objects can also be referred to as samples, examples, instances, data points, or objects. If the data objects are stored in a database, they are data tuples. That is, the rows of a database correspond to the data objects, and the columns correspond to the attributes. In this section, we define attributes and look at the various attribute types.

### 2.1.1 What Is an Attribute?

An attribute is a data field, representing a characteristic or feature of a data object. The nouns attribute, dimension, feature, and variable are often used interchangeably in the literature. The term dimension is commonly used in data warehousing. Machine learning literature tends to use the term feature, while statisticians prefer the term variable. Data mining and database professionals commonly use the term attribute, and we do here as well. Attributes describing a customer object can include, for example, customer ID, name, and address. Observed values for a given attribute are known as observations. A set of attributes used to describe a given object is called an attribute vector (or feature

vector). The distribution of data involving one attribute (or variable) is called univariate. A bivariate distribution involves two attributes, and so on.

The type of an attribute is determined by the set of possible values—nominal, binary, ordinal, or numeric—the attribute can have. In the following subsections, we introduce each type.

### 2.1.2 Nominal Attributes

Nominal means "relating to names." The values of a nominal attribute are symbols or names of things. Each value represents some kind of category, code, or state, and so nominal attributes are also referred to as categorical. The values do not have any meaningful order. In computer science, the values are also known as enumerations.

Example: Nominal attributes. Suppose that hair color and marital status are two attributes describing person objects. In our application, possible values for hair color are black, brown, blond, red, auburn, gray, and white. The attribute marital status can take on the values single, married, divorced, and widowed. Both hair color and marital status are nominal attributes. Another example of a nominal attribute is occupation, with the values teacher, dentist, programmer, farmer, and so on.

### 2.1.3 Binary Attributes

A binary attribute is a nominal attribute with only two categories or states: 0 or 1, where 0 typically means that the attribute is absent, and 1 means that it is present. Binary attributes are referred to as Boolean if the two states correspond to true and false.

Binary attributes. Given the attribute smoker describing a patient object, 1 indicates that the patient smokes, while 0 indicates that the patient does not. Similarly, suppose the patient undergoes a medical test that has two possible outcomes. The attribute medical test is binary, where a value of 1 means the result of the test for the patient is positive, while 0 means the result is negative.

A binary attribute is symmetric if both of its states are equally valuable and carry the same weight; that is, there is no preference on which outcome should be coded as 0 or 1. One such example could be the attribute gender having the states male and female.

A binary attribute is asymmetric if the outcomes of the states are not equally impor- tant, such as the positive and negative outcomes of a medical test for HIV. By convention, we code the most important outcome, which is usually the rarest one, by 1 (e.g., HIV positive) and the other by 0 (e.g., HIV negative).

### 2.1.4 Ordinal Attributes

An ordinal attribute is an attribute with possible values that have a meaningful order or ranking among them, but the magnitude between successive values is not known.

Example 2.3 Ordinal attributes. Suppose that drink size corresponds to the size of drinks available at a fast-food restaurant. This nominal attribute has three possible values: small, medium, and large. The values have a meaningful sequence (which corresponds to increasing drink size); however, we cannot tell from the values how much bigger, say, a medium is than a large. Other examples of ordinal attributes  professional rank. Professional ranks can be enumerated in a sequential order: for example, assistant, associate, and full for professors, and private, private first class, specialist, corporal, and sergeant for army ranks.

Ordinal attributes are useful for registering subjective assessments of qualities that cannot be measured objectively; thus ordinal attributes are often used in surveys for ratings. In one survey, participants were asked to rate how satisfied they were as cus- tomers. Customer satisfaction had the following ordinal categories: 0: very dissatisfied, 1: somewhat dissatisfied, 2: neutral, 3: satisfied, and 4: very satisfied.

Ordinal attributes may also be obtained from the discretization of numeric quantities by splitting the value range into a finite number of ordered categories as described in Chapter 3 on data reduction.

Note that nominal, binary, and ordinal attributes are qualitative. That is, they describe a feature of an object without giving an actual size or quantity. The values of such qualitative attributes are typically words representing categories. If integers are used, they represent computer codes for the categories, as opposed to measurable quantities (e.g., 0 for small drink size, 1 for medium, and 2 for large).

### 2.1.5 Numeric Attributes

A numeric attribute is quantitative; that is, it is a measurable quantity, represented in integer or real values. Numeric attributes can be interval-scaled or ratio-scaled.

**Interval-Scaled Attributes**

Interval-scaled attributes are measured on a scale of equal-size units. The values of interval-scaled attributes have order and can be positive, 0, or negative. Thus, in addition to providing a ranking of values, such attributes allow us to compare and quantify the difference between values.

**Ratio-Scaled Attributes**

A ratio-scaled attribute is a numeric attribute with an inherent zero-point. That is, if a measurement is ratio-scaled, we can speak of a value as being a multiple (or ratio) of another value. In addition, the values are ordered, and we can also compute the difference between values, as well as the mean, median, and mode.

### 2.1.6 Discrete versus Continuous Attributes

We have organized attributes into nominal, binary, ordinal, and numeric types. There are many ways to organize attribute types. The types are not mutually exclusive.

Classification algorithms developed from the field of machine learning often talk of attributes as being either discrete or continuous. Each type may be processed differently. A discrete attribute has a finite or countably infinite set of values, which may or may not be represented as integers. The attributes hair color, smoker, medical test, and drink size each have a finite number of values, and so are discrete. Note that discrete attributes may have numeric values, such as 0 and 1 for binary attributes or, the values 0 to 110 for the

attribute age. An attribute is countably infinite if the set of possible values is infinite but the values can be put in a one-to-one correspondence with natural numbers. For example, the attribute customer ID is countably infinite. The number of customers can grow to infinity, but in reality, the actual set of values is countable (where the values can be put in one-to-one correspondence with the set of integers). Zip codes are another example.

If an attribute is not discrete, it is continuous. The terms numeric attribute and continuous attribute are often used interchangeably in the literature. (This can be confusing because, in the classic sense, continuous values are real numbers, whereas numeric values can be either integers or real numbers.) In practice, real values are represented using a finite number of digits. Continuous attributes are typically represented as floating-point variables.

## 2.2 Similarity and distance measures

In data mining applications, such as clustering, outlier analysis, and nearest-neighbor classification, we need ways to assess how alike or unalike objects are in comparison to one another. For example, a store may want to search for clusters of customer objects, resulting in groups of customers with similar characteristics (e.g., similar income, area of residence, and age). Such information can then be used for marketing. A cluster is a collection of data objects such that the objects within a cluster are similar to one another and dissimilar to the objects in other clusters. Outlier analysis also employs clustering based techniques to identify potential outliers as objects that are highly dissimilar to others. Knowledge of object similarities can also be used in nearest-neighbor classification schemes where a given object (e.g., a patient) is assigned a class label (relating to, say, a diagnosis) based on its similarity toward other objects in the model. This section presents similarity and dissimilarity measures, which are referred to as measures of proximity. Similarity and dissimilarity are related. A similarity measure for

two objects, i and j, will typically return value 0 if the objects are completely unalike. The higher the similarity value, the greater the similarity between objects. (Typically, a value of 1 indicates complete similarity, that is, the objects are identical.) A dissimilarity measure works the opposite way. It returns a value of 0 if the objects are the same (and therefore, far from being dissimilar). The higher the dissimilarity value, the more dissimilar the two objects are.

### 2.2.1 Data matrix vs. dissimilarity matrix

we looked at ways of studying the central tendency, dispersion, and spread of observed values for some attribute X. Our objects there were one-dimensional, that is, described by a single attribute. In this section, we talk about objects described by multiple attributes. Therefore we need a change in notation. Suppose that we have n objects (e.g., persons, items, or courses) described by p attributes (also called measurements or features, such as age, height, weight, or gender). The objects are x1 =(x11,x12,...,x1p), x2 = (x21,x22,...,x2p), and so on, where xij is the value for object xi of the jth attribute. For brevity, we hereafter refer to object xi as object i. The objects may be tuples in a relational database and are also referred to as data samples or feature vectors.

Main memory-based clustering and nearest-neighbor algorithms typically operate on either of the following two data structures:

- Data matrix (or object-by-attribute structure): This structure stores the n data objects in the form of a relational table or an n-by-p matrix (n objects ×p attributes):

$$\begin{bmatrix} x_{11} & \cdots & x_{1f} & \cdots & x_{1p} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{i1} & \cdots & x_{if} & \cdots & x_{ip} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ x_{n1} & \cdots & x_{nf} & \cdots & x_{np} \end{bmatrix}.$$

Each row corresponds to an object. As part of our notation, we may use f to index through the p attributes.

• Dissimilarity matrix (or object-by-object structure): This structure stores a collection of proximities that are available for all pairs of n objects. It is often represented by an n-by-n table:

$$\begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & & \\ d(n,1) & d(n,2) & \cdots & \cdots & 0 \end{bmatrix},$$

where d(i,j) is the measured dissimilarity or "difference" between objects i and j. In general, d(i,j) is a nonnegative number that is close to 0 when objects i and j are highly similar or "near" each other, and becomes larger the more they differ. Note that d(i,i)= 0; that is, the difference between an object and itself is 0. Furthermore, d(i,j)= d(j,i). (For readability, we do not show the d(j,i) entries since the matrix is symmetric.) Measures of dissimilarity are discussed throughout the remainder of this chapter. Measures of similarity can often be expressed as a function of measures of dissimilarity. For example, for nominal data,

$$sim(i, j) = 1 - d(i, j),$$

Where sim(i,j) is the similarity between objects i and j. Throughout the rest of this chapter, we will also comment on measures of similarity. A data matrix is made up of two entities or "things," namely rows (for objects) and columns (for attributes). Therefore, the data matrix is often called a two-mode matrix. The dissimilarity matrix contains one kind of entity (dissimilarities) and so is called a one-mode matrix. Many

clustering and nearest-neighbor algorithms operate on a dissimilarity matrix. Data in the form of a data matrix can be transformed into a dissimilarity matrix before applying such algorithms.

### 2.2.2 Proximity measures for nominal attributes

A nominal attribute can take on two or more states For example, map_color is a nominal attribute that may have, say, five states: red, yellow, green, pink, and blue. Let the number of states of a nominal attribute be M. The states can be denoted by letters, symbols, or asset of integers, such as 1,2,...,M. Notice that such integers are used just for data handling and do not represent any specific ordering. "How is dissimilarity computed between objects described by nominal attributes?" The dissimilaity between two objects i and j can be computed based on the ratio of mismatches:

$$d(i, j) = \frac{p - m}{p},$$

Where m is the number of matches (i.e., the number of attributes for which I and j are in the same state), and p is the total number of attributes describing the objects. Weights can be assigned to increase the effect of m or to assign greater weight to the matches in attributes having a larger number of states.

**Example. Dissimilarity between nominal attributes**.

Suppose that we have the sample data of Table2.4, except that only the object-identifier and the attribute test-1are available, where test-1 is nominal. (We will use test-2 and test-3 in later examples.) Let's compute the dissimilarity matrix that is,

$$\begin{bmatrix} 0 & & & \\ d(2,1) & 0 & & \\ d(3,1) & d(3,2) & 0 & \\ d(4,1) & d(4,2) & d(4,3) & 0 \end{bmatrix}.$$

Since here we have one nominal attribute, test1, we set p=1 so that d(i,j) evaluates to 0 if objects I and j match, and1if the objects differ. Thus, we get

$$\begin{bmatrix} 0 & & & \\ 1 & 0 & & \\ 1 & 1 & 0 & \\ 0 & 1 & 1 & 0 \end{bmatrix}.$$

From this, we see that all object s are dissimilar except objects 1 and 4 (i.e., d(4,1)=0). Alternatively, similarity can be computed as

$$sim(i, j) = 1 - d(i, j) = \frac{m}{p}.$$

**Table 2.4 A sample data table containing attributes of mixed types.**

| Object Identifier | Test-1 (nominal) | Test-2 (ordinal) | Test-3 (numeric) |
|---|---|---|---|
| 1 | code A | excellent | 45 |
| 2 | code B | fair | 22 |
| 3 | code C | good | 64 |
| 4 | code A | excellent | 28 |

Proximity between objects described by nominal attributes can be computed using an alternative encoding scheme. Nominal attributes can be encoded using asymmetric binary attributes by creating a new binary attribute for each of the M states. For an object with a given state value, the binary attribute representing that state is set to 1, whereas the remaining binary attributes are set to 0. For example, to encode the nominal attribute map_color, a binary attribute can be created for each of the five colors previously listed. For an object having the color yellow, the yellow attribute is set to 1, whereas the remaining four attributes are set to 0. Proximity measures for this form of encoding can be calculated using the methods discussed in the next subsection.

### 2.2.3 Proximity measures for binary attributes

Let's look at dissimilarity and similarity measures for objects described by either symmetric or asymmetric binary attributes. Recall that a binary attribute has only one of two states, 0 and 1, where 0 means that the attribute is absent, and 1 means that it is present Given the attribute smoker describing a patient, for instance, 1 indicates that the patient smokes, whereas 0 indicates that the patient does not. Treating binary attributes as if they are other numeric attributes can be misleading. Therefore methods specific to binary data are necessary for computing dissimilarity. "So, how can we compute the dissimilarity between two binary attributes?" One approach involves computing a dissimilarity matrix from the given binary data. If all binary attributes are thought of as having the same weight, we have the 2×2 contingency table of Table 2.5, where q is the number of attributes that equal 1 for both objects i and j, r is the number of attributes that equal 1 for object i but equal 0 for object j, s is the number of attributes that equal 0 for object i but equal 1 for object j, and t is the number of attributes that equal 0 for both objects i and j. The total number of attributes is p, where p = q +r +s+t.

Recall that for symmetric binary attributes, each state is equally valuable. Dissimilarity that is based on symmetric binary attributes is called symmetric binary dissimilarity. If objects i and j are described by symmetric binary attributes, then the dissimilarity between i and j is

$$d(i, j) = \frac{r + s}{q + r + s + t}.$$

For asymmetric binary attributes, the two states are not equally important, such as the positive (1) and negative (0) outcomes of a disease test. Given two asymmetric binary attributes, the agreement of two 1s (a positive match) is then considered more significant than that of two 0s (a negative match). Therefore such binary attributes are often considered "monary" (having one state). The dissimilarity

**Table 2.5 Contingency table for binary attributes.**

| | | Object $j$ | | |
|---|---|---|---|---|
| | | 1 | 0 | sum |
| Object $i$ | 1 | $q$ | $r$ | $q+r$ |
| | 0 | $s$ | $t$ | $s+t$ |
| | sum | $q+s$ | $r+t$ | $p$ |

based on these attributes is called asymmetric binary dissimilarity, where the number of negative matches, t, is considered unimportant and is thus ignored in the following computation:

$$d(i, j) = \frac{r+s}{q+r+s}.$$

Complementarily, we can measure the difference between two binary attributes based on the notion of similarity instead of dissimilarity. For example, **the asymmetric binary similarity** between the objects i and j can be computed as

$$sim(i, j) = \frac{q}{q+r+s} = 1 - d(i, j).$$

The coefficient sim(i,j) of Eq. (2.20) is called the Jaccard coefficient and is popularly referenced in the literature.

When both symmetric and asymmetric binary attributes occur in the same data set, the approach for mixed attributes can be applied.

Example . Dissimilarity between binary attributes. Suppose that a patient record table (Table 2.6) contains the attributes name, gender, fever, cough, test-1, test-2, test-3, and test-4, where name is an object identifier, gender is a symmetric binary attribute, and the remaining attributes are asymmetric binary. For asymmetric binary attribute values, let the values Y (yes) and P (positive) be set to 1,

and the value N (no or negative) be set to 0. Suppose that the distance between objects (patients) is computed based only on the asymmetric binary attributes, the distance between each pair of the three patients—Jack, Mary, and Jim—is

$$d(Jack, Jim) = \frac{1+1}{1+1+1} = 0.67,$$
$$d(Jack, Mary) = \frac{0+1}{2+0+1} = 0.33,$$
$$d(Jim, Mary) = \frac{1+2}{1+1+2} = 0.75.$$

These measurements suggest that Jim and Mary are unlikely to have a similar disease because they have the highest dissimilarity value among the three pairs. Of the three patients, Jack and Mary are the most likely to have a similar disease.

**Table 2.6 Relational table where patients are described by binary attributes.**

| Name | Gender | Fever | Cough | Test-1 | Test-2 | Test-3 | Test-4 |
|---|---|---|---|---|---|---|---|
| Jack | M | Y | N | P | N | N | N |
| Jim | M | Y | Y | N | N | N | N |
| Mary | F | Y | N | P | N | P | N |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |

## 2.2.4 Dissimilarity of numeric data: Minkowski distance

In this section, we describe distance measures that are commonly used for computing the dissimilarity of objects described by numeric attributes. These measures include the Euclidean, Manhattan, and Minkowski distances.

In some cases, the data are normalized before applying distance calculations. This involves transforming the data to fall within a smaller or common range, such as [−1.0,1.0] or [0.0, 1.0]. Consider a height attribute, for example, which could be measured in either meters or inches.

In general, expressing an attribute in smaller units will lead to a larger range for that attribute and thus tend to give such attributes greater effect or "weight." Normalizing the data attempts to give all attributes an equal weight. It may or may not be useful in a particular application.

**Methods for normalizing data.**

The most popular distance measure is **Euclidean distance** (i.e., straight line or "as the crow flies").

Let i $=(x_{i1},x_{i2},...,x_{ip})$ and j $= (x_{j1},x_{j2},...,x_{jp})$ be two objects described by p numeric attributes.

**The Euclidean distance between objects i and j is defined as**

$$d(i, j) = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \cdots + (x_{ip} - x_{jp})^2}.$$

Another well-known measure is the **Manhattan (or city block) distance**, named so because it is the distance in blocks between any two points in a city (such as 2 blocks down and 3 blocks over for a total of 5 blocks). It is defined as

$$d(i, j) = |x_{i1} - x_{j1}| + |x_{i2} - x_{j2}| + \cdots + |x_{ip} - x_{jp}|.$$

Both the Euclidean and the Manhattan distance satisfy the following mathematical properties:

- **Nonnegativity**: $d(i,j) \geq 0$: Distance is a nonnegative number.
- **Identity of indiscernibles**: $d(i,i)= 0$: The distance of an object to itself is 0.
- **Symmetry**: $d(i,j)=d(j,i)$: Distance is a symmetric function.
- **Triangle inequality**: $d(i,j) \leq d(i,k)+d(k,j)$: Going directly from object i to object j in space is no more than making a detour over any other object k.

A measure that satisfies these conditions is known as metric. Please note that the nonnegativity property is implied by the other three properties.

The **supremum distance** (also referred to as $L_{max}$, $L_\infty$ norm, and the **Chebyshev distance**) is a generalization of the Minkowski distance for h →∞. To compute it, we find the attribute f that gives the maximum difference in values between the two objects. This difference is the supremum distance, defined more formally as:

$$d(i, j) = \lim_{h \to \infty} \left( \sum_{f=1}^{p} |x_{if} - x_{jf}|^h \right)^{\frac{1}{h}} = \max_{f}^{p} |x_{if} - x_{jf}|.$$

The $L_\infty$ norm is also known as the uniform norm.

### 2.2.5 Proximity measures for ordinal attributes

The values of an ordinal attribute have a meaningful order or ranking about them, yet the magnitude between successive values is unknown. An example includes the sequence small, medium, large for a size attribute. Ordinal attributes may also be obtained from the discretization of numeric attributes by splitting the value range into a finite number of categories. These categories are organized into ranks. That is, the range of a numeric attribute can be mapped to an ordinal attribute f having $M_f$ states.

For example, the range of the interval-scaled attribute temperature (in Celsius) can be organized into the following states: −30 to −10, −10 to 10, and 10 to 30, representing the categories cold temperature, moderate temperature, and warm temperature, respectively. Let $M_f$ represent the number of

Possible states that an ordinal attribute can have. These ordered states define the ranking1,...,$M_f$.

*"How are ordinal attributes handled?"* The treatment of ordinal attributes is quite similar to that of numeric attributes when computing dissimilarity between objects.

Suppose that f is an attribute from a set of ordinal attributes describing no bjects. The dissimilarity computation with respect of involves the following steps:

1. The value of f for the ith object is $x_{if}$, and f has $M_f$ ordered states, representing the ranking 1,...,$M_f$. Replace each $x_{if}$ by its corresponding rank,$r_{if} \in \{1,...,M_f\}$.

2. Since each ordinal attribute can have a different number of states, it is often necessary to map the Range of each attribute onto [0.0,1.0] so that each attribute has equal weight. We perform such data Normalization by replacing the rank $r_{if}$ of the $i_{th}$ object in the $f_{th}$ attribute by

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}.$$

3. Dissimilarity can then be computed using any of the distance measures \ for numeric attributes, using $z_{if}$ to represent the f value for the $i_{th}$ object.

### 2.2.7 Cosine similarity

**Cosine similarity** measures the similarity between two vectors of an inner product space. It is measured by the cosine of the angle between two vectors and determines whether two vectors are pointing in roughly the same direction. It is often used to measure document similarity in text analysis.

A document can be represented by thousands of attributes, each recording the frequency of a particular word (such as a keyword) or phrase in the document. Thus each document is an object represented by what is called a term-frequency vector. For example, in Table 2.7, we see that Document 1 contains five instances of the word *team*, whereas *hockey* occurs three times. The word *coach* is absent from the entire document, as indicated by a count value of 0. Such data can be highly asymmetric.

Term-frequency vectors are typically very long and sparse (i.e., they have many 0 values). Applications using such structures include information retrieval, text document

clustering, and biological data analysis. The traditional distance measures that we have studied in this chapter do not work well for such sparse numeric data. For example, two term-frequency vectors may have many 0 values in common, meaning that the corresponding documents do not share many words, but this does not make them similar. We need a measure that will focus on the words that the two documents do have in common, and the occurrence frequency of such words. In other words, we need a measure for numeric data that ignores zero-matches.

Cosine similarity is a measure of similarity that can be used to compare documents or, say, give a ranking of documents with respect to a given vector of query words. Let d1 and d2 be two vectors for comparison. Using the cosine measure as a similarity function, we have

$$\mathbf{Cos\ (d1\ ,\ d2\ ) = (d1 \bullet d2\ ) / \|d1\| \ \|d2\|}$$

where • indicates vector dot product, ||d||: the length of vector d

**Table 2.7 Document vector or term-frequency vector.**

| Document | Team | Coach | Hockey | Baseball | Soccer | Penalty | Score | Win | Loss | Season |
|---|---|---|---|---|---|---|---|---|---|---|
| *Document1* | 5 | 0 | 3 | 0 | 2 | 0 | 0 | 2 | 0 | 0 |
| *Document2* | 3 | 0 | 2 | 0 | 1 | 1 | 0 | 1 | 0 | 1 |
| *Document3* | 0 | 7 | 0 | 2 | 1 | 0 | 0 | 3 | 0 | 0 |
| *Document4* | 0 | 1 | 0 | 0 | 1 | 2 | 2 | 0 | 3 | 0 |

cos(d1 , d2 ) = (d1 • d2 ) /||d1 || ||d2 ||

, where • indicates vector dot product, ||d|: the length of vector d ν Ex:

Find the similarity between documents 1 and 2.

d1 = (5, 0, 3, 0, 2, 0, 0, 2, 0, 0)

d2 = (3, 0, 2, 0, 1, 1, 0, 1, 0, 1)

d1 •d2 = 5*3+0*0+3*2+0*0+2*1+0*1+0*1+2*1+0*0+0*1 = 25

||d1 ||= (5*5+0*0+3*3+0*0+2*2+0*0+0*0+2*2+0*0+0*0)0.5=(42)0.5 = 6.481

||d2 ||= (3*3+0*0+2*2+0*0+1*1+1*1+0*0+1*1+0*0+1*1)0.5=(17)0.5 = 4.12

cos(d1 , d2 ) = 0.94

Exercises

2.1 Suppose that the data for analysis include the attribute age. The age values for the data tuples are (in ascending order) 13, 15, 16, 16, 19, 20, 20, 21, 22, 22, 25, 25, 25, 25, 30, 33, 33, 35, 35, 35, 35, 36, 40, 45, 46, 52, 70.

a. What is the mean of the data? What is the median?

b. What is the mode of the data? Comment on the data's modality (i.e., bimodal, trimodal, etc.).

c. What is the midrange of the data? d. Can you find(roughly) the first quartile (Q1) and the third quartile (Q3) of the data?

e. Give the five-number summary of the data.

f. Show a boxplot of the data.

2.2. Suppose that the values for a given set of data are grouped into intervals. The intervals and corresponding frequencies are as follows:

| Age | Frequency |
|---|---|
| 1–5 | 200 |
| 6–15 | 450 |
| 16–20 | 300 |
| 21–50 | 1500 |
| 51–80 | 700 |
| 81–110 | 44 |

# Chapter 3
# Data preprocessing

Data preprocessing is a crucial step in the data mining and machine learning process. Raw data collected from various sources are often incomplete, inconsistent, and may contain errors or noise. Before meaningful analysis or model training can be performed, the data must be cleaned, transformed, and organized into a suitable format.

The main goal of data preprocessing is to improve the quality of the data and, consequently, the performance and accuracy of analytical models. This stage typically involves several key tasks such as data cleaning, data integration, data transformation, and data reduction. Each of these processes ensures that the data becomes more consistent, reliable, and ready for knowledge discovery.

In short, data preprocessing converts raw data into a well-structured dataset that supports effective analysis and decision-making. It is often said that high-quality input data leads to high-quality outcomes—making preprocessing one of the most important phases in the entire data analysis pipeline.

## 3.1 Data quality, data cleaning, and data integration

In this section, we start with a discussion of data quality measures. Then, we introduce common techniques for data cleaning and data integration.

### 3.1.1 Data quality measures

Data preprocessing is an essential step in data mining and machine learning. Raw data collected from different sources are often noisy, incomplete, or inconsistent. Before we can analyze or model the data, it must be cleaned and prepared properly to ensure good quality and reliable results.

**What Is Data Quality?**

Data quality refers to how well the data meets the requirements of its intended use. In other words, high-quality data should be accurate, complete, consistent, timely, believable, and easy to interpret. These characteristics can be viewed as **measures for data quality** and are summarized below:

- **Accuracy:** Indicates whether the data values are correct or incorrect.
- **Completeness:** Shows if some data are missing or unavailable.
- **Consistency:** Ensures that data do not contain contradictions or mismatched values.
- **Timeliness:** Refers to how up-to-date the data are and whether they are updated in time.
- **Believability:** Describes how much users can trust that the data are correct and reliable.
- **Interpretability:** Represents how easily the data can be understood and used by others.

In real-world databases, it is common to find data that are inaccurate, incomplete, or inconsistent due to human errors, technical issues, or missing information. Such problems reduce the quality of the data and make analysis difficult.

Therefore, **data preprocessing** plays a critical role in improving data quality. It includes processes such as **data cleaning, integration, transformation, and reduction**, which help convert raw data into a consistent and useful form for further analysis and knowledge discovery.

## 3.1.2 Data cleaning

Real-world data tend to be incomplete, noisy, and inconsistent. Data cleaning (or data cleansing) routines attempt to fill in missing values, smooth out noise while identifying outliers, and correct in consistencies in the data. In this section, you will study basic

methods for data cleaning. First, we look at ways of handling missing values. Then, we explain data smoothing techniques. Finally, we discuss approaches to data cleaning as a process.

**Missing values**

Imagine that you need to analyze the sales and customer data of a company. You note that many tuples have no recorded value for several attributes such as customer income. How can you go about filling in the missing values for this attribute? Let's look at the following methods.

**1. Ignore the tuple:** This is usually done when the class label is missing (assuming the mining task involves classification). This method is not very effective, unless the tuple contains several attributes with missing values. It is especially poor when the percentage of missing values per attribute varies considerably. By ignoring the tuple, we do not make use of the remaining attributes' values in the tuple. Such data could have been useful to the task at hand.

**2. Fill in the missing value manually:** In general, this approach is time consuming and may not be feasible given a large data set with many missing values.

**3. Use a global constant to fill in the missing value:** Replace all missing attribute values by the same constant such as a label like "Unknown" or $-\infty$. If missing values are replaced by, say, "Unknown," then the mining program may mistakenly think that they form an interesting concept, since they all have a value in common—that of "Unknown." Hence, although this method is simple, it is not foolproof.

**4. Use a measure of central tendency for the attribute (e.g., the mean or median) to fill in the missing value:** which indicate the "middle" value of a data distribution. For normal (symmetric) data distributions, the mean can be used, whereas skewed data distribution should employ the median, For example, suppose that the data distribution

regarding the income of the customers is symmetric and that the mean income is $56,000. Use this value to replace the missing value for income.

**5. Use the attribute mean or median for all samples belonging to the same class as the given tuple:** For example, if classifying customers according to credit_risk, we may replace the missing value with the mean income value for customers in the same credit risk category as that of the given tuple. If the data distribution for a given class is skewed, the median value is a better choice.

**6. Use the most probable value to fill in the missing value:** This may be determined with regression, inference-based tools using a Bayesian formalism or decision tree induction. For example, using the other customer attributes in your data set, you may construct a decision tree to predict the missing values for income.

Methods 3 through 6 bias the data—the filled-in value may not be correct. Method 6, however, is a popular strategy. In comparison to the other methods, it uses the most information from the present data to predict missing values. By considering the values of other attributes in its estimation of the missing value for income, there is a greater chance that the relationships between income and the other attributes are preserved. It is important to note that, in some cases, a missing value may not imply an error in the data! For example, when applying for a credit card, candidates may be asked to supply their driver's license number. Candidates who do not have a driver's license may naturally leave this field blank. Forms should allow respondents to specify values such as "not applicable." Software routines may also be used to uncover other null values (e.g., "don't know," "?", or "none"). Ideally, each attribute should have one or more rules regarding the null condition. The rules may specify whether or not nulls are allowed and/or how such values should be handled or transformed. Fields may also be intentionally left blank if they are to be provided in a later step of the business process. Hence, although we can try our best to clean the data after it is seized, good database and

data entry procedure design should help minimize the number of missing values or errors in the first place.

### 3.1.3 Noisy data

"What is noise?" Noise is a random error or variance in a measured variable. Given a numeric attribute such as, say, price, how can we "smooth" out the data to remove the noise? Let's look at the following data smoothing techniques.

**Binning:** Binning methods smooth a sorted data value by consulting its "neighborhood," that is, the values around it. The sorted values are distributed into a number of "buckets," or bins. Because binning methods consult the neighborhood of values, they perform local smoothing. Fig. 2.11 illustrates some binning techniques. In this example, the data for price are first sorted and then partitioned into equal-frequency bins of size 3 (i.e., each bin contains three values). In smoothing by bin means, each value in a bin is replaced by the mean value of the bin. For example, the mean of the values 4, 8, and 15 in Bin 1 is 9. Therefore each original value in this bin is replaced by the value 9. Similarly, smoothing by bin medians can be employed, in which each bin value is replaced by the bin median. In smoothing by bin boundaries, the minimum and maximum values in a given bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value. In general, the larger the width, the greater the effect of the smoothing. Alternatively, bins may be

**Sorted data for price (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34**

**Partition into (equal-frequency) bins**:
Bin 1: 4, 8, 15
Bin 2: 21, 21, 24
Bin 3: 25, 28, 34

**Smoothing by bin means**:
Bin 1: 9, 9, 9
Bin 2: 22, 22, 22
Bin 3: 29, 29, 29

**Smoothing by bin boundaries**:
Bin 1: 4, 4, 15
Bin 2: 21, 21, 24
Bin 3: 25, 25, 34

**FIGURE 2.11 Data smoothing with different binning methods.**

Equal width, where the interval range of values in each bin is constant. Binning is also used as a discretization technique.

**Regression:** Data smoothing can also be done by regression, a technique that conforms data values to a function. Linear regression involves finding the "best" line to fit two attributes (or variables) so that one attribute can be used to predict the other. Multiple linear regression is an extension of linear regression, where more than two attributes are involved, and the data are fit to a multidimensional surface.

**Outlier analysis:** Outliers may be detected by clustering, for example, where similar values are organized into groups or "clusters." Intuitively, values that fall outside of the set of clusters may be considered as outliers.

## 3.2 Data Integration

Data integration means combining data from multiple sources into one unified view or data store, such as a data warehouse. It helps remove redundancies and inconsistencies, improving both the accuracy and efficiency of data analysis.

However, integration is not always easy because data may differ in structure, format, or meaning across sources. The main challenges include:

1. **Entity Identification Problem**

This occurs when similar data from different sources use different names or formats.

For example, customer_id in one database and cust_number in another may represent the same attribute. Metadata (information about data type, meaning, range, etc.) can help match and merge these attributes correctly.

2. **Redundancy and Correlation Analysis**

Sometimes the same information appears in different forms, causing redundancy.

For example, annual revenue may be derived from monthly sales.

Statistical methods such as the chi-square test (for categorical data) or correlation coefficient (for numeric data) can help detect relationships and redundancies between attributes.

3. **Tuple Duplication**

Duplicate records may exist in a dataset due to errors or denormalized tables.

For instance, the same customer might appear with slightly different addresses in several entries. Removing duplicate tuples helps ensure data consistency.

4. **Data Value Conflict Detection and Resolution**

Sometimes the same real-world entity has different values across systems — for example:

One system stores weight in kilograms, another in pounds. Prices may vary due to different currencies or tax rules. Such conflicts must be identified and resolved by using standard units, scales, or transformation rules before integration.

## 3.3 Data Transformation

**Data transformation** is the process of converting data into suitable formats for data mining. It helps make the mining process **more efficient** and the discovered patterns **easier to interpret**. This step often involves changing, scaling, or grouping data so that it becomes more consistent and ready for analysis. The main techniques include:

1. **Data Normalization**

Normalization scales numeric values into a smaller, standard range such as **0 to 1** or **–1 to 1**. It ensures that all attributes contribute equally to the analysis, especially when they have different units or scales (for example, "age" vs. "income").

2. **Data Discretization**

Discretization replaces continuous numeric values with **intervals** or **categories**. For example, the attribute *age* can be grouped into ranges like *0–10, 11–20, 21–30*, or into conceptual labels like *youth, adult, senior*.

This helps simplify the data and makes patterns easier to understand.

3. **Data Compression**

Compression reduces the data size while preserving important information. It helps improve storage efficiency and speed up data mining.

4. **Data Sampling**

Sampling selects a smaller representative subset from a large dataset. It reduces processing time while maintaining the essential patterns and relationships within the data.

# Chapter 4

# Data warehousing and online analytical processing

Data analytics, also often known as business intelligence, is the strategies and technology that enable enterprises to gain deep and actionable insights into business data. Data mining plays the core role in data analytics and business intelligence. Fundamentally, data warehouses generalize and consolidate data in multidimensional space. The construction of data warehouses involves data cleaning, data integration, and data transformation, and can be viewed as an important preparation step for data mining. Moreover, data warehouses provide online analytical processing (OLAP) tools for interactive analysis of multidimensional data of varied granularities, which facilitates effective data generalization and data mining. Many other data mining functions, such as association, classification, prediction, and clustering, can be integrated with OLAP operations to enhance interactive mining of knowledge at multiple levels of abstraction. OLAP tools typically use data cube, a multidimensional data model, to provide flexible access to summarized data. Data lakes as enterprise information infrastructure collect extensive data in enterprises and integrate metadata so that data exploration can be conducted effectively. Hence, data warehouses, OLAP, data cubes, and data lakes have become essential data and information backbone for enterprises. This chapter presents an in-depth and comprehensive introduction to data warehouse, OLAP, data cube, and data lake technology. This overview is essential for understanding the overall data mining and knowledge discovery process and practical applications. In addition, it can serve as a well-informed introduction to data analytics and business intelligence.

### 4.1 Relationship between Business Analytics (BA), Business Intelligence (BI), and Data Warehouse (DW)

**Data Warehouse (DW)**, **Business Intelligence (BI)**, and **Business Analytics (BA)** are closely related concepts that work together to support data-driven decision making within organizations.

A **Data Warehouse** serves as the foundation. It is a centralized repository that integrates data from multiple sources, cleans and organizes it, and stores it in a consistent format. The data warehouse provides the historical and current data needed for analysis.

**Business Intelligence (BI)** builds on top of the data warehouse. It focuses on **descriptive and diagnostic analysis**, such as generating reports, dashboards, and visualizations that summarize business performance. BI tools help users understand *what happened* and *why it happened* by transforming warehouse data into meaningful insights.

**Business Analytics (BA)** goes a step further by applying **statistical, predictive, and machine learning techniques** to forecast future trends and recommend actions. BA answers questions like *what will happen* and *what should we do next*.

In summary:

- The **Data Warehouse** provides the data foundation.
- **Business Intelligence** transforms that data into useful information for decision support.
- **Business Analytics** uses advanced analysis to predict and optimize future outcomes.

Together, these three components form a complete data-driven ecosystem that helps organizations make better, faster, and smarter business decisions.

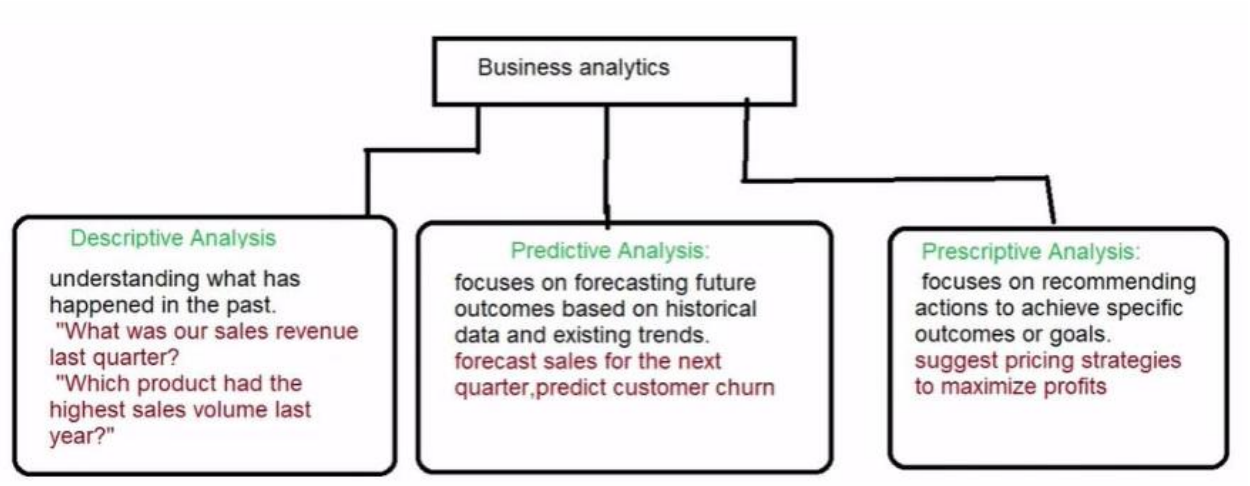



**Figure 4.1 types of business analytics**

## 4.2 What is a Data Warehouse?

**Data warehouse** is a pool of data produced to support decision making; it is also a repository of **current** and **historical** data of potential interest to managers throughout the organization.

- Data are usually structured to be available in a form ready for analytical processing activities (i.e., online analytical processing [OLAP], data mining, querying, reporting, and other decision support applications).
- "A data warehouse is a subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management's decision-making process."

➢ **Data Warehouse—Subject-Oriented**

Provide a simple and concise view around particular subject issues by excluding data that are not useful in the decision support process

➢ **Data Warehouse—Integrated**

Constructed by integrating multiple, heterogeneous data sources, relational databases, flat files, on-line transaction records

- **Data Warehouse—Time Variant**

The time horizon for the data warehouse is significantly longer than that of operational systems

• Operational database: current value data

• Data warehouse data: provide information from a historical perspective (e.g., past 5-10 years)

- **Data Warehouse—Nonvolatile**

  - A physically separate store of data transformed from the operational environment
  - Operational update of data does not occur in the data warehouse environment
  - Does not require transaction processing, recovery, and concurrency control mechanisms
  - Requires only two operations in data accessing: ν initial loading of data and access of data

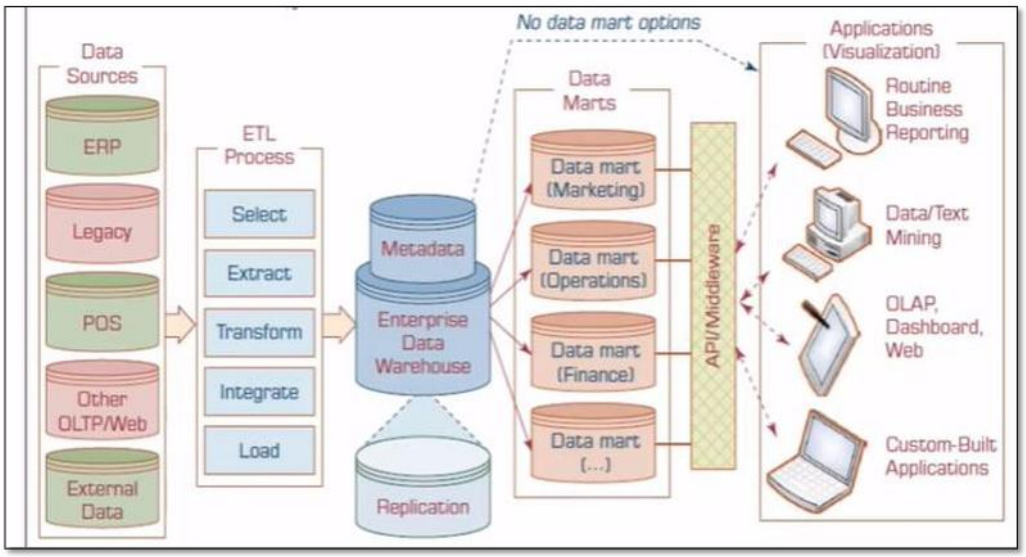



**Figure 4.2 datawrehouse framework**

## 4.3 ETL Process (Extract, Transform, Load)

The **ETL process** is a fundamental step in building and maintaining a **Data Warehouse (DW)**. It stands for **Extract, Transform, and Load**, describing the three main stages of moving data from multiple sources into a unified and organized data warehouse.

**1. Extract**

In this step, data are **collected (extracted)** from various sources such as databases, files, spreadsheets, or online systems. These sources often store data in different formats or structures. The main goal of extraction is to gather all the relevant data needed for analysis without altering their original meaning.

**2. Transform**

Once extracted, the data are **transformed** into a consistent format suitable for analysis. This stage may include:

- **Cleaning** the data (removing errors, duplicates, or missing values).
- **Converting formats or units** (e.g., changing currencies or date formats).
- **Integrating** data from different systems.
- **Applying business rules** or calculations to make the data meaningful.

Transformation ensures that the data are accurate, consistent, and ready to be loaded into the warehouse.

**3. Load**

Finally, the cleaned and transformed data are **loaded** into the **data warehouse**. This can be done in two ways:

- **Full loading:** where all data are loaded at once (usually during initial setup).
- **Incremental loading:** where only new or updated data are added periodically.

After the loading process, the data warehouse becomes ready for **Business Intelligence (BI)** and **Business Analytics (BA)** applications such as reporting, visualization, and predictive modeling.

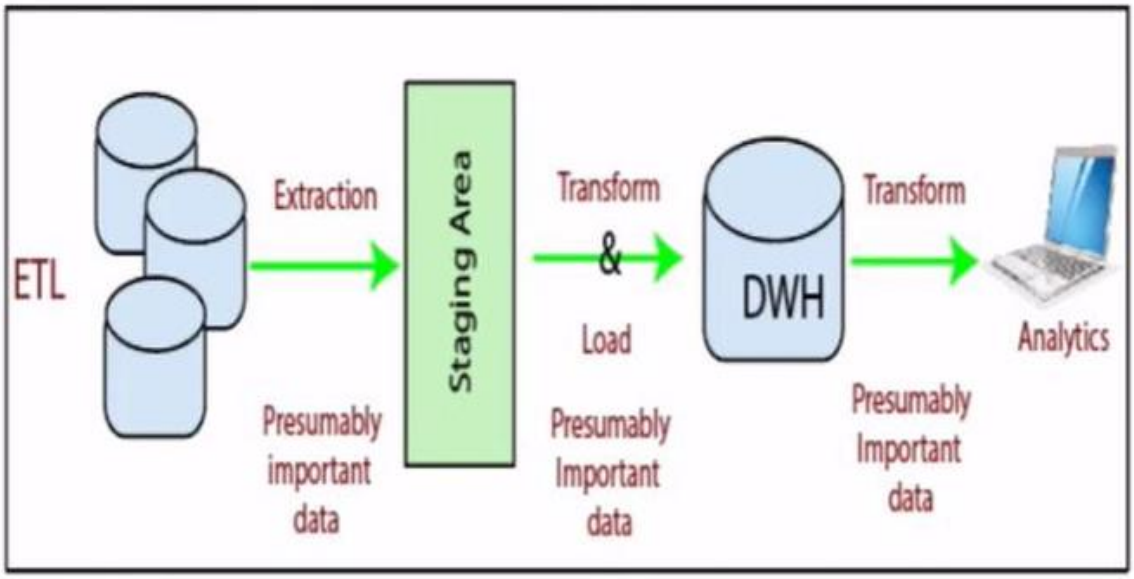



Figure 4.3 Extract, Transform, Load (ETL)

## 4.4 Transaction Processing vs Analytic Processing

Transaction Processing Systems are constantly involved in handling updates to what we might call operational databases.

- For example, in an ATM withdrawal transaction, we need to reduce our bank balance accordingly; a bank deposit adds to an account; and a grocery store purchase is likely reflected in the store's calculation of total sales for the day, and it should reflect an appropriate reduction in the store's inventory for the items we bought, and so on
- Online Transaction Processing (OLTP) systems handle a company's routine ongoing business.
- In contrast, a DW is typically a distinct system that provides storage for data that will be used for analysis. So, DWs are intended to work with informational data used for online analytical processing (OLAP) systems.
- OLTP is a term used for a transaction system that is primarily responsible for capturing and storing data related to day-to-day business functions such as ERP, CRM, SCM, POS, and so forth

- An OLTP system addresses a critical business need, automating daily business transactions, and running real-time reports and routine analysis. But these systems are not designed for ad hoc analysis and complex queries that deal with a number of data items. OLAP, on the other hand, is designed to address this need by providing ad hoc analysis of organizational data much more effectively and efficiently.
- OLAP and OLTP rely heavily on each other: OLAP uses the data captured by OLTP, and OLTP automates the business processes that are managed by decisions supported by OLAP.

**Table 4.1 OLTP vs OLAP**

| | **OLTP** | **OLAP** |
|---|---|---|
| **users** | clerk, IT professional | knowledge worker |
| **function** | day to day operations | decision support |
| **DB design** | application-oriented | subject-oriented |
| **data** | current, up-to-date<br>detailed, flat relational<br>isolated | historical,<br>summarized, multidimensional<br>integrated, consolidated |
| **usage** | repetitive | ad-hoc |
| **access** | read/write<br>index/hash on prim. key | lots of scans |
| **unit of work** | short, simple transaction | complex query |
| **# records accessed** | tens | millions |
| **#users** | thousands | hundreds |
| **DB size** | 100MB-GB | 100GB-TB |
| **metric** | transaction throughput | query throughput, response |

## 4.5 Key Differences between Normalization and Denormalization

- Normalization is the method of arranging the data in the database efficiently. It involves constructing tables and setting up relationships between those tables according to some certain rules. The redundancy and inconsistent dependency can be removed using these rules in order to make it more flexible.
- There are 6 defined normal forms: 1NF, 2NF, 3NF, BCNF, 4NF and 5NF. Normalization should eliminate the redundancy but not at the cost of integrity.
- Denormalization is the inverse process of normalization, where the normalized schema is converted into a schema which has redundant information.
- The performance is improved by using redundancy and keeping the redundant data consistent.
- The reason for performing Denormalization is the overheads produced in query processor by an over-normalized structure.
- Normalization is the technique of dividing the data into multiple tables to reduce data redundancy and inconsistency and to achieve data integrity. On the other hand, Denormalization is the technique of combining the data into a single table to make data retrieval faster.
- Normalization is used in OLTP system, which emphasizes on making the insert, delete and update anomalies faster. As against, Denormalization is used in OLAP system, which emphasizes on making the search and analysis faster.
- Normalization is the technique of dividing the data into multiple tables to reduce data redundancy and inconsistency and to achieve data integrity. On the other hand, Denormalization is the technique of combining the data into a single table to make data retrieval faster.

- Normalization is used in OLTP system, which emphasizes on making the insert, delete and update anomalies faster. As against, Denormalization is used in OLAP system, which emphasizes on making the search and analysis faster.

## 4.6 Data Warehouse Models: Enterprise Warehouse, Data Mart, and Virtual Warehouse

When designing a Data Warehouse environment, an organization may adopt one of three fundamental models, depending on its analytical needs, data volume, business structure, and cost considerations. These models differ in scope, architecture, level of integration, and implementation complexity.

**1. Enterprise Data Warehouse (EDW)**

An **Enterprise Data Warehouse (EDW)** is a **centralized and comprehensive repository** that stores data for the entire organization. It integrates data from various operational systems and business units to provide a unified, consistent, and enterprise-wide view of information.

**Key Characteristics:**

- Integrates data from all departments and operational systems across the enterprise.
- Supports strategic decision-making and organization-wide analytics.
- Maintains **historical data** to facilitate trend analysis, forecasting, and long-term planning.
- Ensures data consistency and accuracy by providing a **single version of the truth**.

**Advantages:**

- Provides a holistic and unified data view.
- Enhances data consistency, quality, and governance.
- Supports advanced analytics, Business Intelligence (BI), OLAP, and Data Mining applications.

**Limitations:**

- Requires significant time, cost, and resources to design and implement.
- Complex to maintain due to its large scale and centralized nature.

**2. Data Mart**

A **Data Mart** is a **subject-oriented, smaller-scale** data warehouse designed for a specific business unit, department, or functional area within the organization (such as Marketing, Finance, HR, or Sales).

**Key Characteristics:**

- Focuses on a specific business process or department.
- Easier and faster to build compared to an EDW.
- May be independent (sourced directly from operational systems) or dependent (sourced from an EDW).

**Advantages:**

- Lower cost, faster implementation, and simpler maintenance.
- Provides tailored analytics that meet departmental needs.
- Enhances performance due to smaller data volume.

**Limitations:**

- May lead to data inconsistency if multiple isolated Data Marts are created.
- Limited enterprise-wide integration unless aligned with an EDW.

**3. Virtual Data Warehouse (VDW)**

A **Virtual Warehouse** is a logical (virtual) view of the data warehouse that provides access to data stored in different locations without physically consolidating it into a central repository.

**Key Characteristics:**

- Provides a **virtual integration layer**, typically through metadata and views.
- Data remains in source systems and is accessed in real-time or near real-time for analysis.

- Often implemented using middleware, federated databases, or virtualization technologies.

**Advantages:**

- Low cost and rapid deployment since no physical storage is required.
- Flexible and scalable for organizations not ready for a full EDW implementation.
- Real-time access to up-to-date data.

**Limitations:**

- Performance may be slower because data is queried from multiple sources.
- Complex to manage if source systems have inconsistent structures or data quality issues.
- Limited historical data storage compared to a physical EDW.

## 4.7 Data Cube: A Multidimensional Data Mode

"What is a data cube?" A data cube allows data to be modeled and viewed in multiple dimensions. It is defined by dimensions and facts. In general terms, dimensions are the perspectives or entities with respect to which an organization wants to keep records. For example, AllElectronics may create a sales data warehouse in order to keep records of the store's sales with respect to the dimensions time, item, branch, and location. These dimensions allow the store to keep track of things like monthly sales of items and the branches and locations at which the items were sold. Each dimension may have a table associated with it, called a dimension table, which further describes the dimension. For example, a dimension table for item may contain the attributes item name, brand, and type. Dimension tables can be specified by users or experts, or automatically generated and adjusted based on data distributions. A multidimensional data model is typically organized around a central theme, such as sales. This theme is represented by a fact table. Facts are numeric measures. Think of them as the quantities by which we want to analyze relationships between dimensions. Examples of facts for a sales data warehouse

include dollars sold (sales amount in dollars), units sold (number of units sold), and amount budgeted. The fact table contains the names of the facts, or measures, as well as keys to each of the related dimension tables. You will get a clearer picture of how this works when we look at multidimensional schemas. Although we usually think of cubes as 3-D geometric structures, in data warehousing the data cube is n-dimensional. To gain a better understanding of data cubes and the multidimensional data model, let's start by looking at a simple 2-D data cube that is, in fact, a table or spreadsheet for sales data from AllElectronics. In particular, we will look at the AllElectronics sales data for items sold per quarter in the city of Vancouver. These data are shown in figure 4.4. In this 2-D representation, the sales for Vancouver are shown with respect to the time dimension (organized in quarters) and the item dimension (organized according to the types of items sold). The fact or measure displayed is dollars sold (in thousands).

| | ***location*** = *"Vancouver"* | | | |
|---|---|---|---|---|
| | ***item*** *(type)* | | | |
| ***time*** *(quarter)* | *home entertainment* | *computer* | *phone* | *security* |
| Q1 | 605 | 825 | 14 | 400 |
| Q2 | 680 | 952 | 31 | 512 |
| Q3 | 812 | 1023 | 30 | 501 |
| Q4 | 927 | 1038 | 38 | 580 |

*Note:* The sales are from branches located in the city of Vancouver. The measure displayed is *dollars_sold* (in thousands).

**Figure 4.4 : 2-D View of Sales Data for AllElectronics According to time and item.**

Now, suppose that we would like to view the sales data with a third dimension. For instance, suppose we would like to view the data according to time and item, as well as location, for the cities Chicago, New York, Toronto, and Vancouver. These 3-D data are shown in figure 4.5. The 3-D data in the table are represented as a series of 2-D tables.

Conceptually, we may also represent the same data in the form of a 3-D data cube, as in Figure 4.6.

| | location = "Chicago" | | | | location = "New York" | | | | location = "Toronto" | | | | location = "Vancouver" | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | item | | | | item | | | | item | | | | item | | | |
| | home | | | | home | | | | home | | | | home | | | |
| time | ent. | comp. | phone | sec. | ent. | comp. | phone | sec. | ent. | comp. | phone | sec. | ent. | comp. | phone | sec. |
| Q1 | 854 | 882 | 89 | 623 | 1087 | 968 | 38 | 872 | 818 | 746 | 43 | 591 | 605 | 825 | 14 | 400 |
| Q2 | 943 | 890 | 64 | 698 | 1130 | 1024 | 41 | 925 | 894 | 769 | 52 | 682 | 680 | 952 | 31 | 512 |
| Q3 | 1032 | 924 | 59 | 789 | 1034 | 1048 | 45 | 1002 | 940 | 795 | 58 | 728 | 812 | 1023 | 30 | 501 |
| Q4 | 1129 | 992 | 63 | 870 | 1142 | 1091 | 54 | 984 | 978 | 864 | 59 | 784 | 927 | 1038 | 38 | 580 |

*Note:* The measure displayed is *dollars_sold* (in thousands).

**Figure. 4.5: 3-D View of Sales Data for AllElectronics According to time, item, and location.**

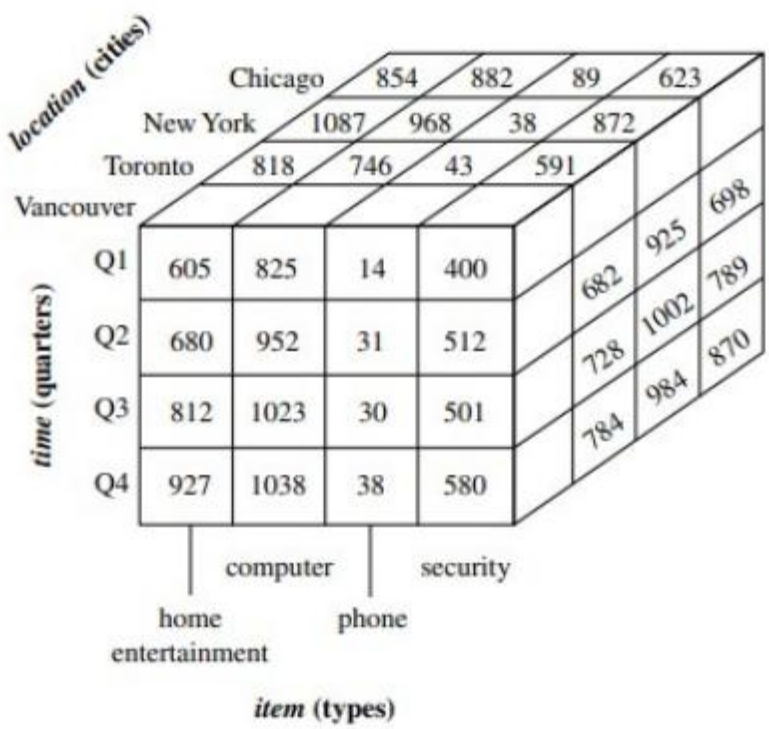



**Figure 4.6: Figure 2.3: A 3-D data cube representation**

Suppose that we would now like to view our sales data with an additional fourth dimension such as supplier. Viewing things in 4-D becomes tricky. However, we can think of a 4-D cube as being a series of 3-D cubes, as shown in Figure4.7

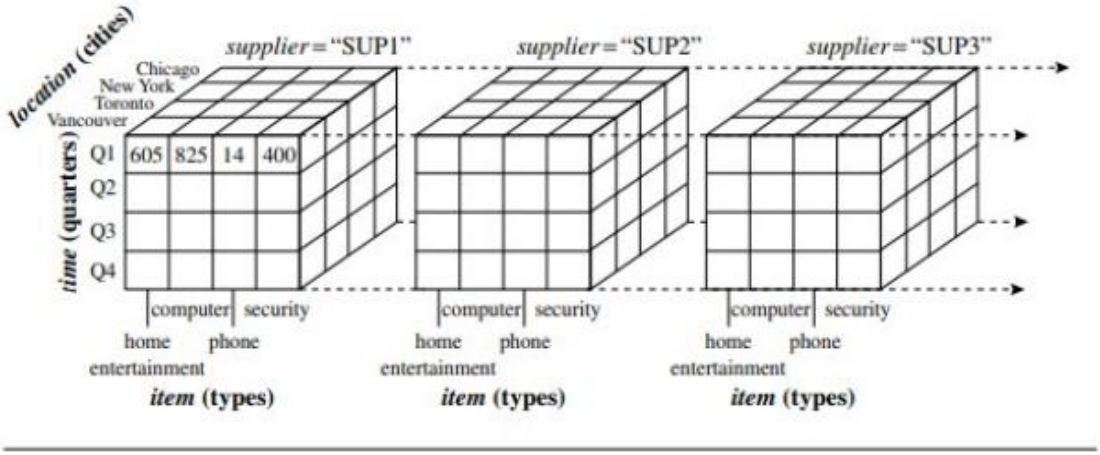



**Figure 4.7: A 4-D data cube representation of sales data, according to time, item, location, and supplier. The measure displayed is dollars sold (in thousands). For improved readability, only some of the cube values are shown.**

Suppose that we would now like to view our sales data with an additional fourth dimension such as supplier. Viewing things in 4-D becomes tricky. However, we can think of a 4-D cube as being a series of 3-D cubes, as shown in Figure 4.7. If we continue in this way, we may display any n-dimensional data as a series of (n 1)-dimensional "cubes." The data cube is a metaphor for multidimensional data storage. The actual physical storage of such data may differ from its logical representation. The important thing to remember is that data cubes are n-dimensional and do not confine data to 3-D. figure 4.4 and figure 4.5 show the data at different degrees of summarization. In the data warehousing research literature, a data cube like those shown in Figures 4.6 and 4.7 is often referred to as a cuboid. Given a set of dimensions, we can generate a cuboid for each of the possible subsets of the given dimensions. The result would form a lattice of cuboids, each showing the data at a different level of

summarization, or group-by. The lattice of cuboids is then referred to as a data cube. Figure 4.8 shows a lattice of cuboids forming a data cube for the dimensions time, item, location, and supplier. The cuboid that holds the lowest level of summarization is called the base cuboid. For example, the 4-D cuboid in Figure 4.7 is the base cuboid for the given time, item, location, and supplier dimensions. Figure 4.6 is a 3-D (nonbase) cuboid for time, item, and location, summarized for all suppliers. The 0-D cuboid, which holds the highest level of summarization, is called the apex cuboid. In our example, this is the total sales, or dollars sold, summarized over all four dimensions. The apex cuboid is typically denoted by all.

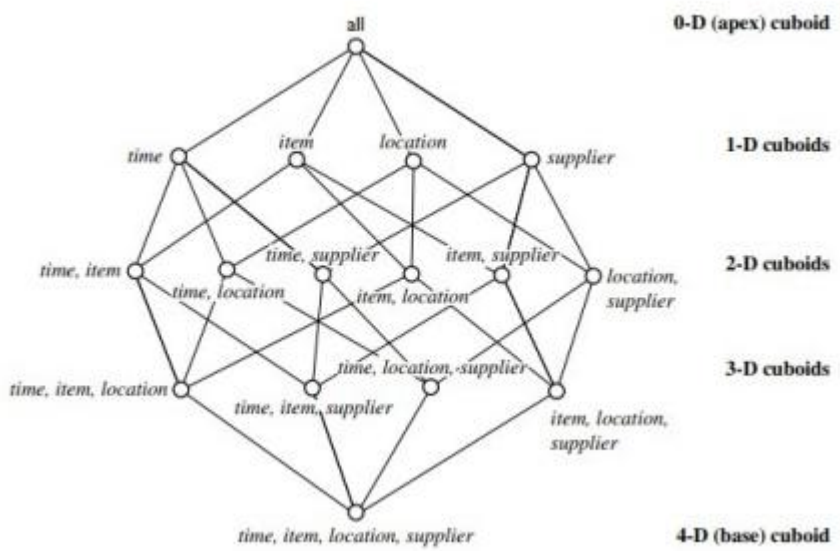



**Figure 4.8: Lattice of cuboids, making up a 4-D data cube for time, item, location, and supplier. Each cuboid represents a different degree of summarization.**

## 4.8 Stars, Snowflakes, and Fact Constellations: Schemas for Multidimensional Data Models

The entity-relationship data model is commonly used in the design of relational databases, where a database schema consists of a set of entities and the relationships between them. Such a data model is appropriate for online transaction processing. A data warehouse, however, requires a concise, subject-oriented schema that facilitates online data analysis. The most popular data model for a data warehouse is a multidimensional model, which can exist in the form of a star schema, a snowflake schema, or a fact constellation schema. Let's look at each of these.

### 4.8.1 Star schema

The most common modeling paradigm is the star schema, in which the data warehouse contains (1) a large central table (fact table) containing the bulk of the data, with no redundancy, and (2) a set of smaller attendant tables (dimension tables), one for each dimension. The schema graph resembles a starburst, with the dimension tables displayed in a radial pattern around the central fact table. Example shown in Figure 4.9

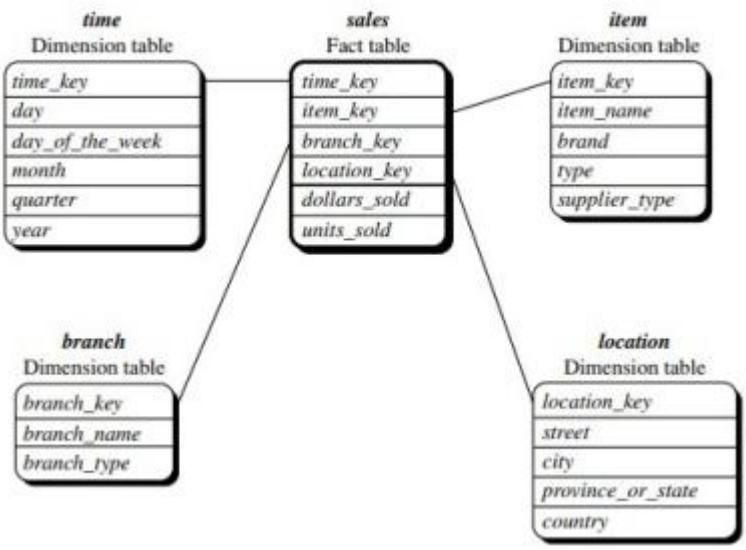



**Figure 4.9: Star schema of sales data warehouse.**

### 4.8.2 Snowflake schema

The snowflake schema is a variant of the star schema model, where some dimension tables are normalized, thereby further splitting the data into additional tables. The resulting schema graph forms a shape similar to a snowflake. The major difference between the snowflake and star schema models is that the dimension tables of the snowflake model may be kept in normalized form to reduce redundancies. Such a table is easy to maintain and saves storage space. However, this space savings is negligible in comparison to the typical magnitude of the fact table. Furthermore, the snowflake structure can reduce the effectiveness of browsing, since more joins will be needed to execute a query. Consequently, the system performance may be adversely impacted. Hence, although the snowflake schema reduces redundancy, it is not as popular as the star schema in data warehouse design. Example 2 Snowflake schema. A snowflake schema for AllElectronics sales is given in Figure 4.10.

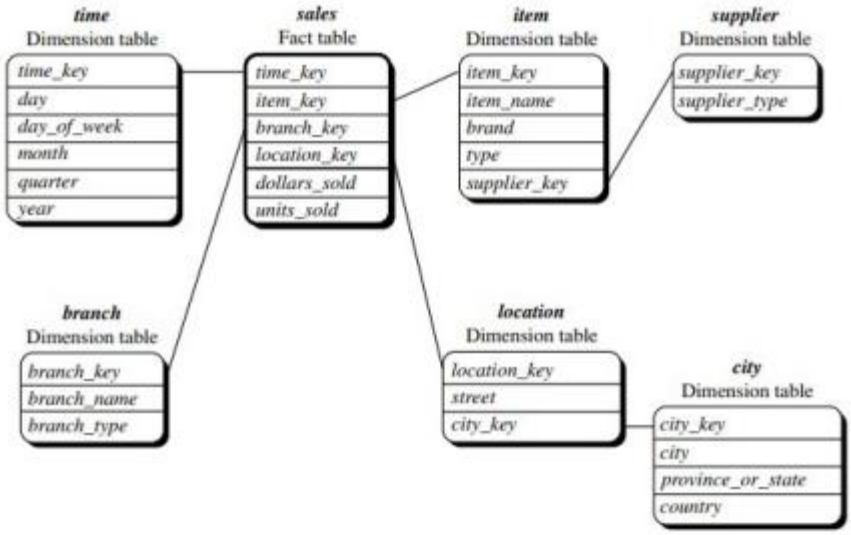



**Figure 4.10: Snowflake schema of a sales data warehouse.**

### 4.8.3 Fact constellation

Sophisticated applications may require multiple fact tables to share dimension tables. This kind of schema can be viewed as a collection of stars, and hence is called a galaxy schema or a fact constellation.

Example 3 Fact constellation. A fact constellation schema is shown in Figure 4.11

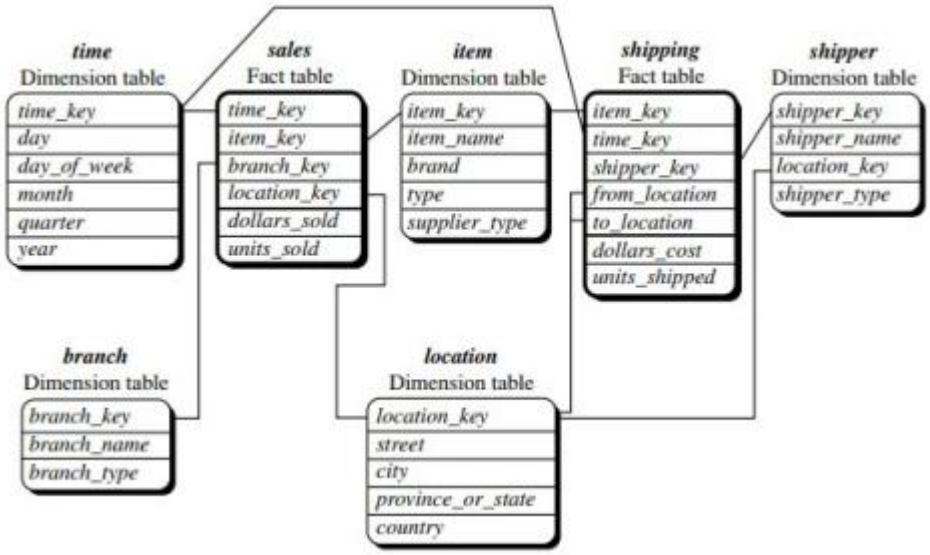



**Figure 4.11: Fact constellation schema of a sales and shipping data warehouse**

In data warehousing, there is a distinction between a data warehouse and a data mart. A data warehouse collects information about subjects that span the entire organization, such as customers, items, sales, assets, and personnel, and thus its scope is enterprise-wide. For data warehouses, the fact constellation schema is commonly used, since it can model multiple, interrelated subjects. A data mart, on the other hand, is a department subset of the data warehouse that focuses on selected subjects, and thus its scope is department wide. For data marts, the star or snowflake schema is commonly used, since both are geared toward modeling single subjects, although the star schema is more popular and efficient.

## 4.9 OLAP Operations

A data warehouse needs to support online multidimensional analytic queries. In this section, you will learn a series of typical OLAP operations on data warehouses.

### 2.3.1 Typical OLAP operations

"How can multidimensional OLAP operations be used in data analysis?" In a multidimensional model, data are organized into multiple dimensions, and each dimension contains multiple levels of abstraction defined by concept hierarchies. This organization provides users with the flexibility to view data from different perspectives. A number of OLAP data cube operations empower interactive querying and analysis of the data at hand. Hence, OLAP provides a user-friendly environment for interactive data analysis.

**Roll-up:**

The roll-up operation (also called the drill-up operation by some vendors) performs aggregation on a data cube, either by climbing up a concept hierarchy for a dimension or by dimension reduction. This hierarchy was defined as the total order street < city < province or state < country. The roll-up operation shown aggregates the data by ascending the location hierarchy from the level of city to the level of country. In other words, rather than grouping the data by city, the resulting cube groups the data by country. When roll-up is performed by dimension reduction, one or more dimensions are removed from the given cube. For example, consider a sales data cube containing only the location and time dimensions. Roll-up may be performed by removing, say, the location dimension, resulting in an aggregation of the total sales by time of the whole company, rather than by location and by time.

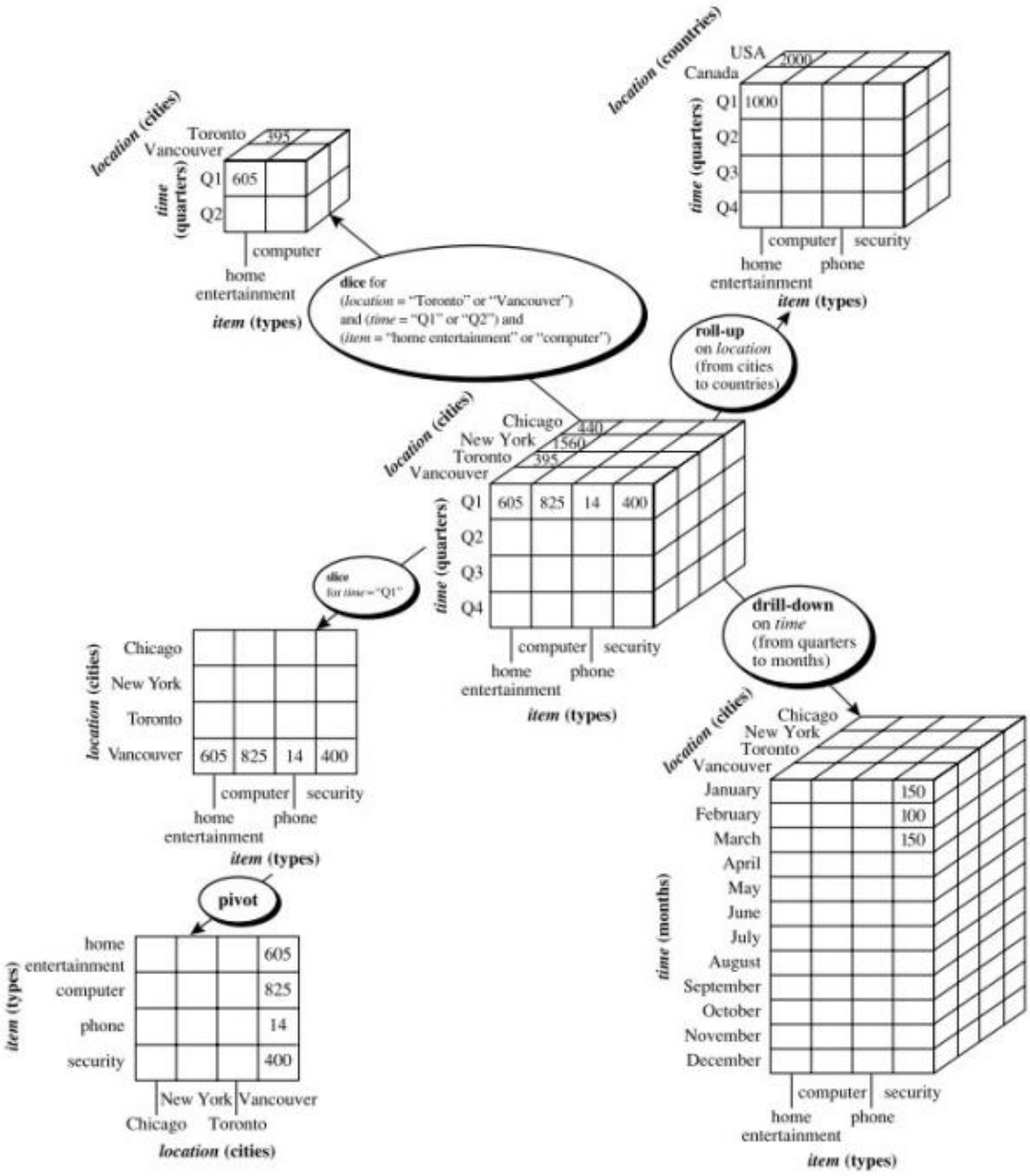



**Figure 4.12: Examples of typical OLAP operations on multidimensional data.**

**Drill-down:**

Drill-down is the reverse of roll-up. It navigates from less detailed data to more detailed data. Drill-down can be realized by either stepping down a concept hierarchy for a dimension or introducing additional dimensions.

**Slice and dice:**

The slice operation performs a selection on one dimension of the given cube, resulting in a subcube. Figure4.12 shows a slice operation where the sales data are selected from the central cube for the dimension time using the criterion time = “Q1.” The dice operation defines a subcube by performing a selection on two or more dimensions. Figure4.12 shows a dice operation on the central cube based on the following selection criteria that involve three dimensions: (location = "Toronto" or "Vancouver") and (time = "Q1" or "Q2") and (item = "home entertainment" or "computer").

**Pivot (rotate):**

Pivot (also called rotate) is a visualization operation that rotates the data axes in view to provide an alternative data presentation. Figure4.12 shows a pivot operation where the item and location axes in a 2-D slice are rotated. Other examples include rotating the axes in a 3-D cube, or transforming a 3-D cube into a series of 2-D planes.

**Example:**

- Suppose that a data warehouse consists of the three dimensions time, doctor, and patient, and the two measures count and charge, where charge is the fee that a doctor charges a patient for a visit.

- Starting with the base cuboid [day, doctor, patient], what specific OLAP operations should be performed in order to list the total fee collected by each doctor in 2010?

# Chapter 5

# Pattern mining: basic concepts and methods

Frequent patterns are patterns (e.g., item sets, subsequences, or substructures) that appear frequently in a data set. For example, a set of items, such as milk and bread, that appear frequently together in a transaction data set is a frequent item set. A subsequence, such as buying first a smartphone, then a smart TV, and then a smart home device, if it occurs frequently in a shopping history database, is a (frequent) sequential pattern. A substructure can refer to different structural forms, such as subgraphs, subtrees, or sublattices. If a substructure occurs frequently, it is called a (frequent) structured pattern. Finding frequent patterns plays an essential role in mining associations, correlations, and many other interesting relationships among data. Moreover, it helps in data classification, clustering, and other data mining tasks. Thus, frequent pattern mining has become an important data mining task and a focused theme in data mining research.

## 5.1 Basic Concepts

Frequent pattern mining uncovers recurring relationships in a given data set. This section introduces the basic concepts of frequent pattern mining for the discovery of interesting associations and correlations between itemsets in transactional and relational databases.

### 5.1.1 Market Basket Analysis

Market Basket Analysis (MBA) is a data mining technique used to discover **associations and co-occurrence patterns** among items frequently purchased together by customers. Each group of purchased items is called an **itemset**. By analyzing transactional data (e.g., shopping baskets), companies can identify relationships between products and use this insight to enhance decision-making in areas such as **store layout design, cross-selling, promotions, catalog design, and customer behavior analysis**.

This technique helps retailers answer questions like:

**“Which items are likely to be purchased together in a single shopping trip?”**

For example, if customers who buy computers also often buy antivirus software, placing these products close to each other may increase combined sales. Alternatively, placing related products in different store sections may encourage customers to pick up additional items while moving through the store.

The discovered associations can be expressed as **association rules**, such as:

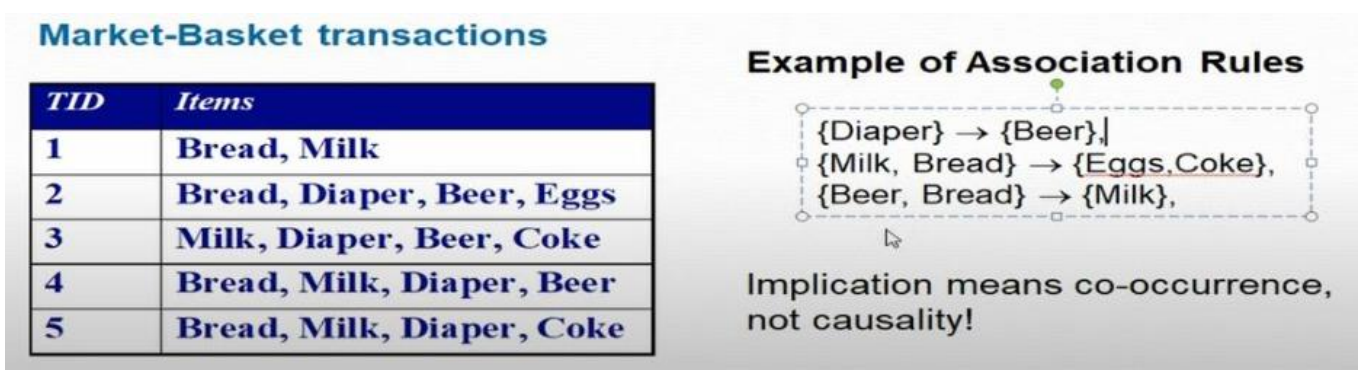
$$computer \Rightarrow antivirus_software \quad [support = 2\%, confidence = 60\%].$$

- Support measures how frequently items appear together in transactions.
- Confidence measures how often the rule is found to be true.

Association rules are considered useful when they satisfy minimum support and minimum confidence thresholds. These thresholds can be tuned by analysts or domain experts to extract the most meaningful patterns.

### 5.2 Association Rule Mining

Given a set of transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction.



**Definition: Frequent Itemset**

• ***Itemset***

- A collection of one or more items
  - Example: {Milk, Bread, Diaper}
- **k-itemset**
  - An itemset that contains *k* items

• ***Support count (σ)***

- Frequency of occurrence of an itemset
  - E.g. σ({Milk, Bread, Diaper}) = 2

• ***Support***

- Fraction of transactions that contain an itemset
  - E.g. s({Milk, Bread, Diaper}) = 2/5

• ***Frequent Itemset***

- An itemset whose support is greater than or equal to a *minsup* threshold

| TID | Items |
|---|---|
| 1 | Bread, Milk |
| 2 | Bread, Diaper, Beer, Eggs |
| 3 | Milk, Diaper, Beer, Coke |
| 4 | Bread, Milk, Diaper, Beer |
| 5 | Bread, Milk, Diaper, Coke |

I assume that itemsets are ordered lexicographically

**Definition: Association Rule**

Let **D** be a database of **transactions**

**E.g:**

| Transaction ID | Items Bought |
|---|---|
| 2000 | A,B,C |
| 1000 | A,C |
| 4000 | A,D |
| 5000 | B,E,F |

- Let **I** be the set of items that appear in the database, e.g.
- **I = {A, B, C, D, E, F}**
- A **rule** is defined by **X ⇒ Y**, where **X ⊂ I**, **Y ⊂ I**, and **X ∩ Y = ∅**
- e.g.: **{B, C} ⇒ {E}** is a rule

**Definition: Association Rule**

***Association Rule***

- An implication expression of the form **X → Y**, where **X** and **Y** are itemsets
- Example:

  **{Milk, Diaper} → {Beer}**

***Rule Evaluation Metrics***

**• Support (s)**

- Fraction of transactions that contain both **X** and **Y**

**• Confidence (c)**

- Measures how often items in **Y** appear in transactions that contain **X**

Example:

$$\{\text{Milk, Diaper}\} \Rightarrow \text{Beer}$$

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{|T|} = \frac{2}{5} = 0.4$$

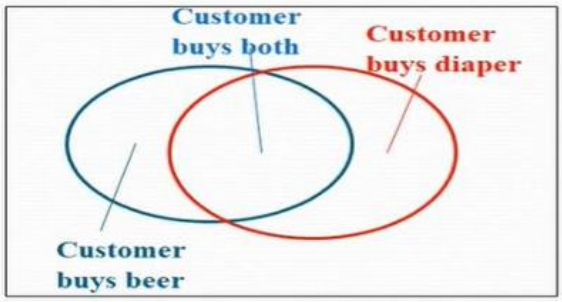
$$c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

| *TID* | *Items* |
|---|---|
| 1 | Bread, Milk |
| 2 | Bread, Diaper, Beer, Eggs |
| 3 | Milk, Diaper, Beer, Coke |
| 4 | Bread, Milk, Diaper, Beer |
| 5 | Bread, Milk, Diaper, Coke |

## Rule Measures: Support and Confidence

Find all the rules **X ⇒ Y** with minimum confidence and support

- **support, s**, probability that a transaction contains **{X ∪ Y}**
- **confidence, c**, conditional probability that a transaction having **X** also contains **Y**




| Transaction ID | Items Bought |
|---|---|
| 2000 | A,B,C |
| 1000 | A,C |
| 4000 | A,D |
| 5000 | B,E,F |

**Let minimum support 50%, and minimum confidence 50%, we have:**

- **A ⇒ C** (50%, 66.6%)
- **C ⇒ A** (50%, 100%)

## Example:

| TID | date | items_bought |
|---|---|---|
| 100 | 10/10/99 | {F,A,D,B} |
| 200 | 15/10/99 | {D,A,C,E,B} |
| 300 | 19/10/99 | {C,A,B,E} |
| 400 | 20/10/99 | {B,A,D} |

Remember:
$$conf(X \Rightarrow Y) = \frac{sup(X \cup Y)}{sup(X)}$$

**What is the support and confidence of the rule: {B, D} ⇒ {A}?**

Support:

Percentage of tuples that contain {A, B, D} = 75%

Confidence:

$$\frac{\text{number of tuples that contain } \{A, B, D\}}{\text{number of tuples that contain } \{B, D\}} = 100\%$$

**Association Rule Mining Task**

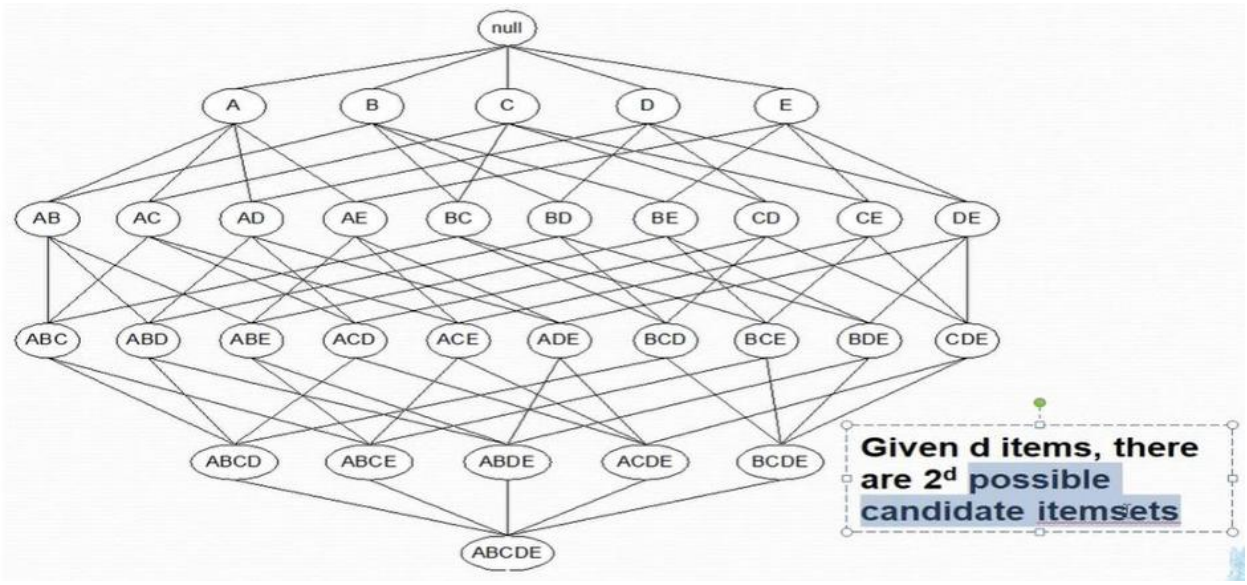
- Given a set of transactions **T**, the goal of association rule mining is to find all rules having
  - **support ≥ minsup** threshold
  - **confidence ≥ minconf** threshold
- **Brute-force approach:**
  - List all possible association rules
  - Compute the support and confidence for each rule
  - Prune rules that fail the **minsup** and **minconf** thresholds

⇒ **Computationally prohibitive!**

## Mining Association Rules

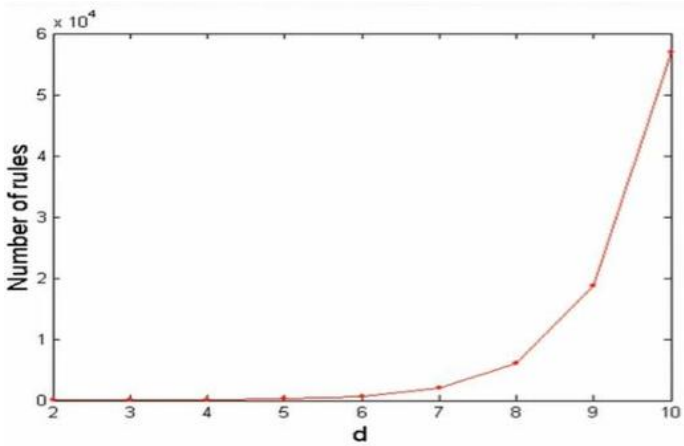
- Two-step approach:
  1. **Frequent Itemset Generation**
     - Generate all itemsets whose support ≥ *minsup*
  2. **Rule Generation**
     - Generate high confidence rules from each frequent *itemset*, where each rule is a binary partitioning of a frequent *itemset*
- **Frequent itemset generation is still computationally expensive**

## 5.3 Frequent itemset generation




### 5.3.1 Computational Complexity

- Given **d** unique items:
  - Total number of itemsets = **2^d**
  - Total number of possible association rules




$$R = \sum_{k=1}^{d-1}\left[\binom{d}{k} \times \sum_{j=1}^{d-k}\binom{d-k}{j}\right]$$
$$= 3^d - 2^{d+1} + 1$$

**If d=6, R = 602 rules**

## Reducing Number of Candidates

***Apriori principle:***

- If an itemset is frequent, then all of its subsets must also be frequent

Apriori principle holds due to the following property of the support measure:

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

- Support of an itemset never exceeds the support of its subsets
- This is known as the **anti-monotone** property of support

## 5.4 Frequent Itemset Generation Strategies

**Reduce the number of candidates (M):**

- Complete search: M = 2^d
- Use pruning techniques to reduce M

**Reduce the number of transactions (N):**

- Reduce size of N as the size of itemset increases
- Used by DHP and vertical-based mining algorithms

**Reduce the number of comparisons (NM):**

- Use efficient data structures to store the candidates or transactions
- No need to match every candidate against every transaction

### 5.4.1 Reducing Number of Candidates

**The downward closure property of frequent patterns**

- **Any subset of a frequent itemset must be frequent**
  - If **{beer, diaper, nuts}** is frequent, so is **{beer, diaper}**
  - i.e., every transaction having {beer, diaper, nuts} also contains {beer, diaper}

**Apriori principle holds due to the following property of the support measure:**

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

 Support of an itemset never exceeds the support of its subsets.

 This is known as the **anti-monotone property** of support

## 5.5 Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested!
- Method:
  - Initially, scan DB once to get frequent 1-itemset
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
  - Test the candidates against DB
  - Terminate when no frequent or candidate set can be generated.

**EXAMPLE:**

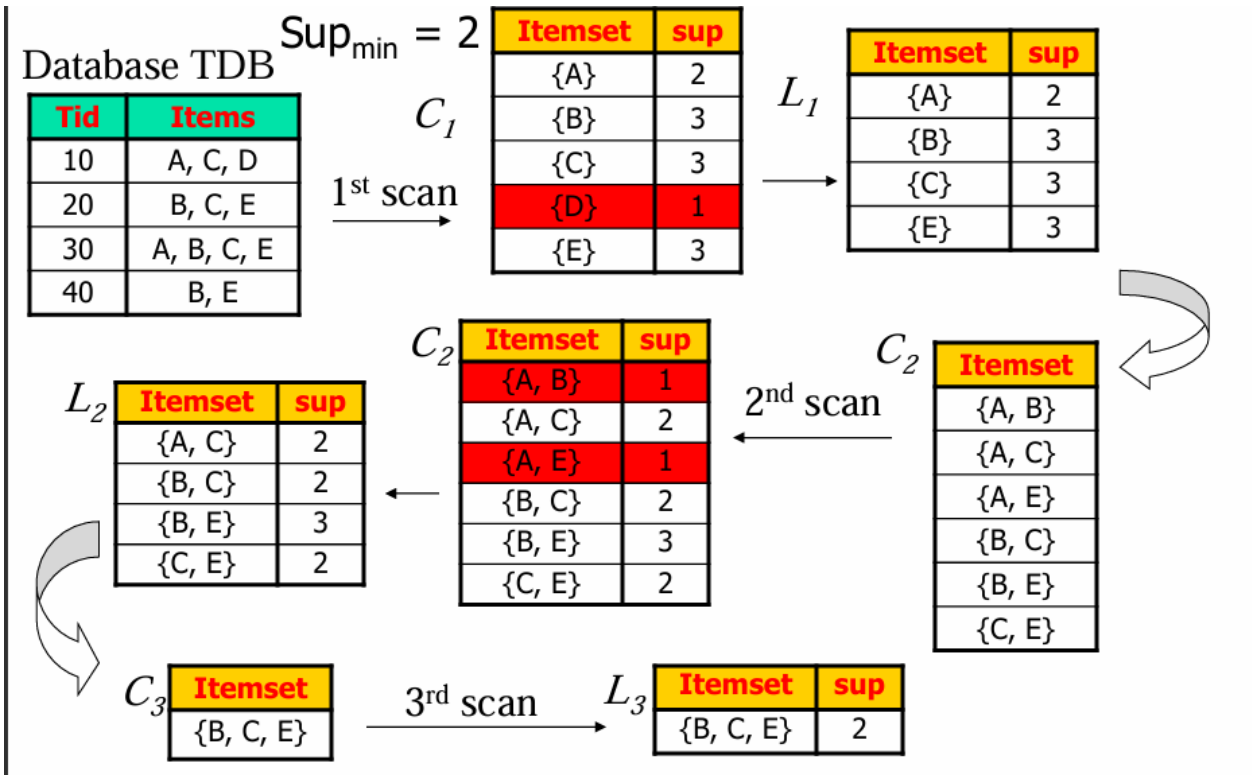



**Figure: 5.1. Apriori algorithm steps showing the generation of candidate and frequent itemsets (C1,L1,C2,L2,C3,L3C_1, L_1, C_2, L_2, C_3, L_3C1,L1,C2,L2,C3,L3) with minimum support = 2.**

### 5.5.1 Implementation of Apriori

**How to generate candidates?**

- **Step 1:** self-joining $L_k$
- **Step 2:** pruning

**Example of Candidate-generation**

- L3={abc,abd,acd,ace,bcd}
- **Self-joining:** L3∗L3
  - abcd from **abc** and **abd**
  - acde from **acd** and **ace**

- **Pruning:**
  - Acde is removed because **ade** is not in L3
- C4={abcd}

### 5.5.2 Maximal Frequent Itemset

An itemset is maximal frequent if none of its immediate supersets is frequent.

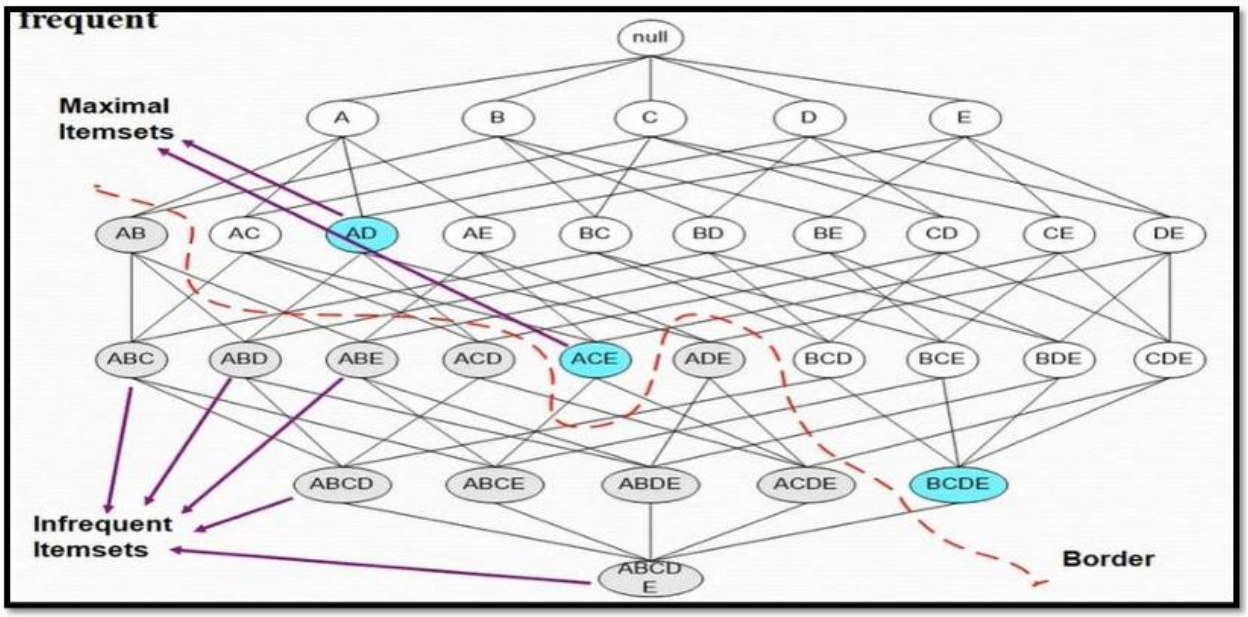



**Figure: 5.2. Lattice diagram showing frequent, infrequent, and maximal itemsets, with the border separating frequent from infrequent itemsets**

### 5.5.3 Closed Itemset

- An itemset is closed if none of its immediate supersets has the same support as the itemset.

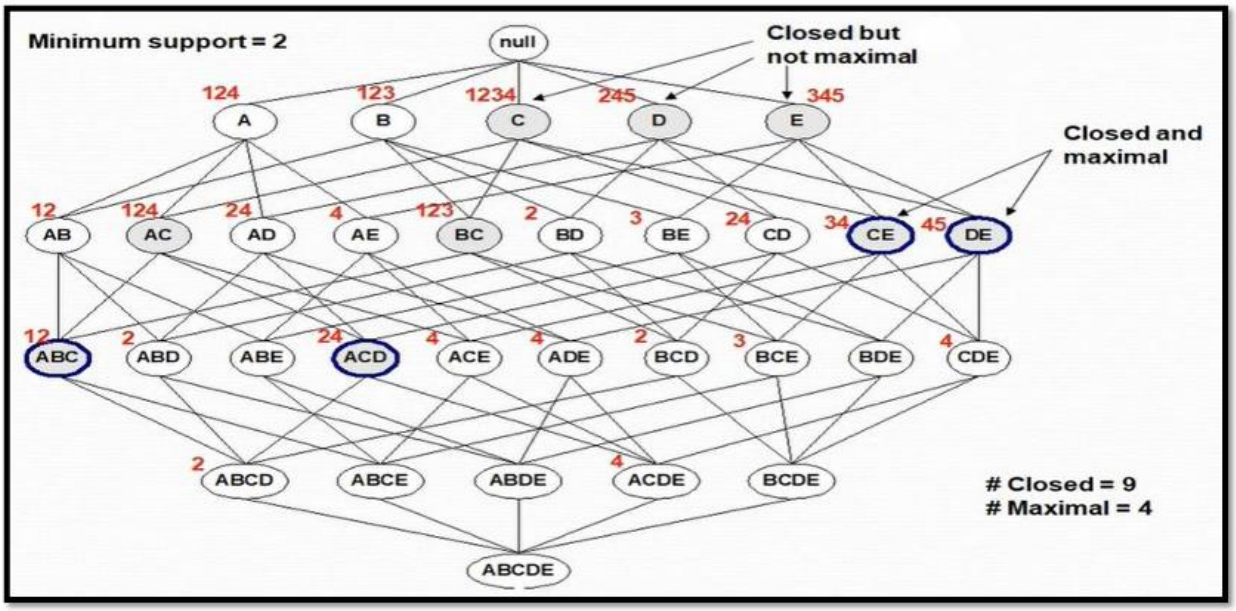



**Figure: 5.3. Closed vs. maximal itemsets in a lattice with minimum support = 2**

## 5.6 Factors Affecting Complexity

- **Choice of minimum support threshold**

Lowering support threshold results in more frequent itemsets.

This may increase the number of candidates and the maximum length of frequent itemsets.

- **Dimensionality (number of items) of the data set**

More space is needed to store the support count of each item.

If the number of frequent items also increases, both computation and I/O costs may also increase.

- **Size of database**

Since Apriori makes multiple passes, the runtime of the algorithm may increase with the number of transactions.

- **Average transaction width**

Transaction width increases with denser data sets. This may increase the maximum length of frequent
itemsets and traversals of the hash tree (number of subsets in a transaction increases with its width).

**RULE GENERATION**

- Given a frequent itemset L, find all non-empty subsets $f \subset L$ such that $f \rightarrow L - f$ satisfies the minimum confidence requirement
  - If {A,B,C,D} is a frequent itemset, candidate rules:

| | | | |
|---|---|---|---|
| ABC →D, | ABD →C, | ACD →B, | BCD →A, |
| A →BCD, | B →ACD, | C →ABD, | D →ABC |
| AB →CD, | AC → BD, | AD → BC, | BC →AD, |
| BD →AC, | CD →AB, | | |

- If $|L| = k$, then there are $2^k - 2$ candidate association rules (ignoring $L \rightarrow \varnothing$ and $\varnothing \rightarrow L$)

How to efficiently generate rules from frequent itemsets?

- In general, confidence does not have an anti-monotone property
  c(ABC→D) can be larger or smaller than c(AB→D)
- But confidence of rules generated from the same itemset has an anti-monotone property

- e.g., L = {A,B,C,D}:

$$c(ABC \rightarrow D) \geq c(AB \rightarrow CD) \geq c(A \rightarrow BCD)$$

- Confidence is anti-monotone w.r.t. number of items on the RHS of the rule.

## FP-growth: Mining Frequent Patterns Without Candidate Generation

- Compress a large database into a compact, **Frequent-Pattern tree (FP-tree) structure**
  - Highly condensed, but complete for frequent pattern mining
  - Avoid costly database scans
- Develop an efficient, FP-tree-based frequent pattern mining method
  - A divide-and-conquer methodology: decompose mining tasks into smaller ones
  - Avoid candidate generation: sub-database test only

## 5.7 FP-tree Construction from a Transactional DB

| TID | Items bought | (ordered) frequent items |
|---|---|---|
| 100 | {f, a, c, d, g, i, m, p} | {f, c, a, m, p} |
| 200 | {a, b, c, f, l, m, o} | {f, c, a, b, m} |
| 300 | {b, f, h, j, o, w} | {f, b} |
| 400 | {b, c, k, s, p} | {c, b, p} |
| 500 | {a, f, c, e, l, p, m, n} | {f, c, a, m, p} |

*min_support = 3*

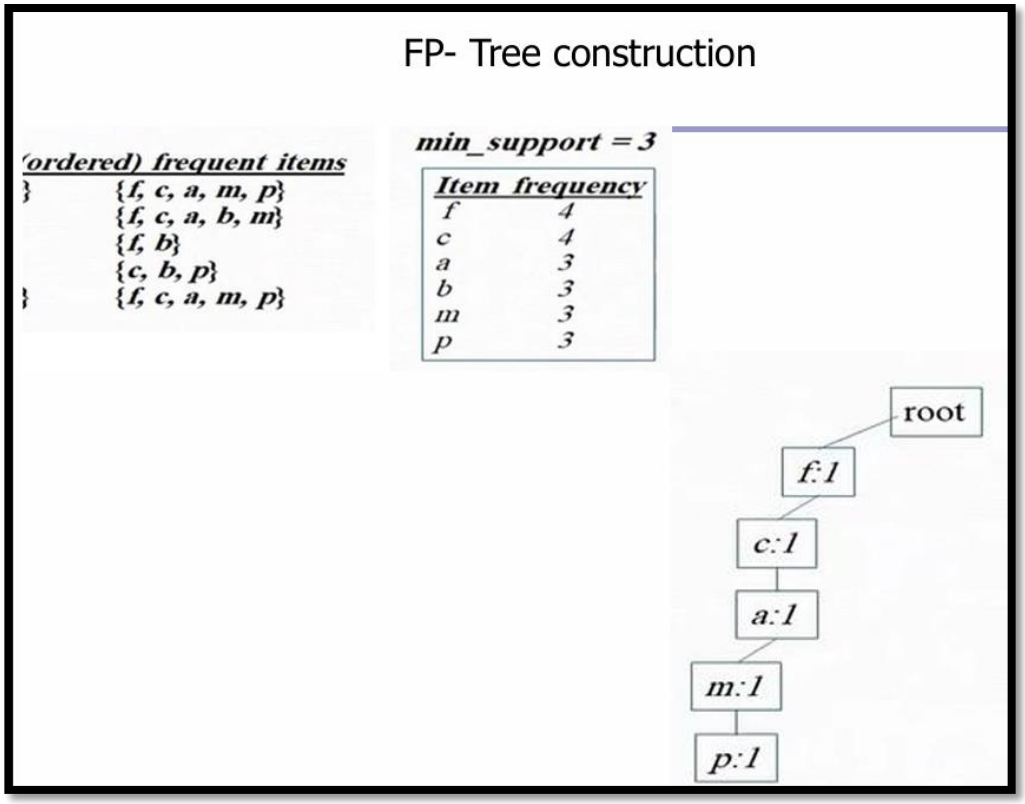
| Item | frequency |
|---|---|
| f | 4 |
| c | 4 |
| a | 3 |
| b | 3 |
| m | 3 |
| p | 3 |

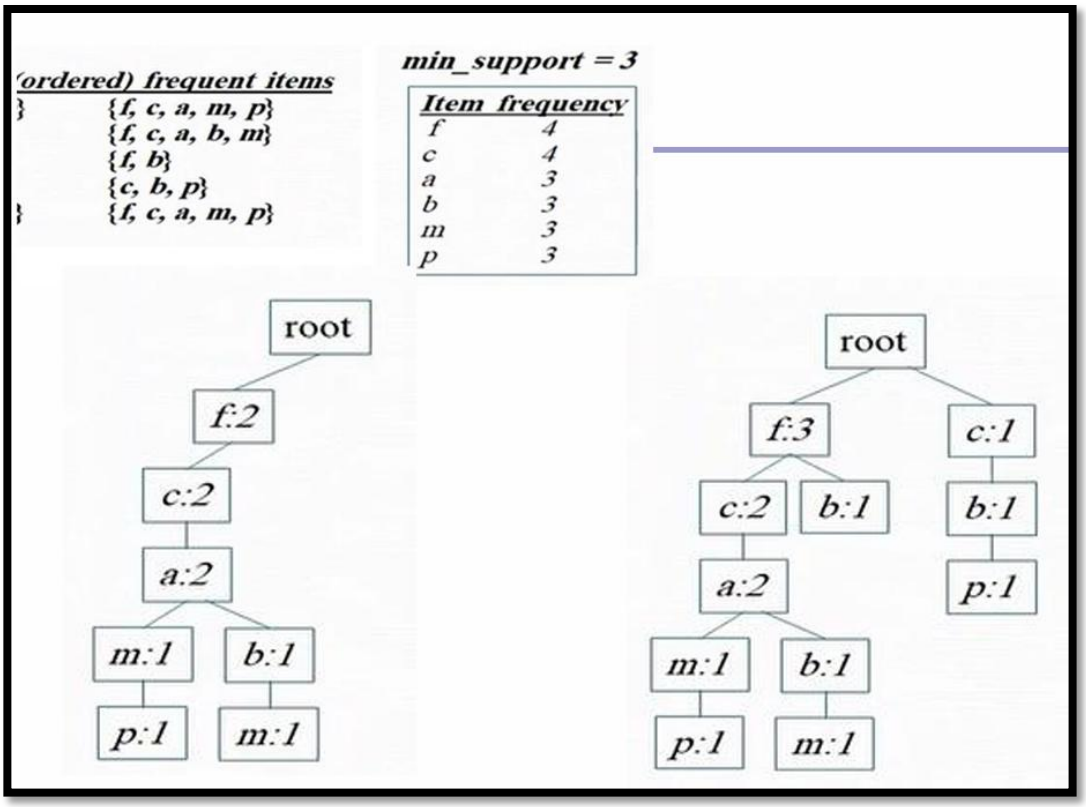
**Steps:**

1- Scan DB once, find frequent 1-itemsets( single item patterns)

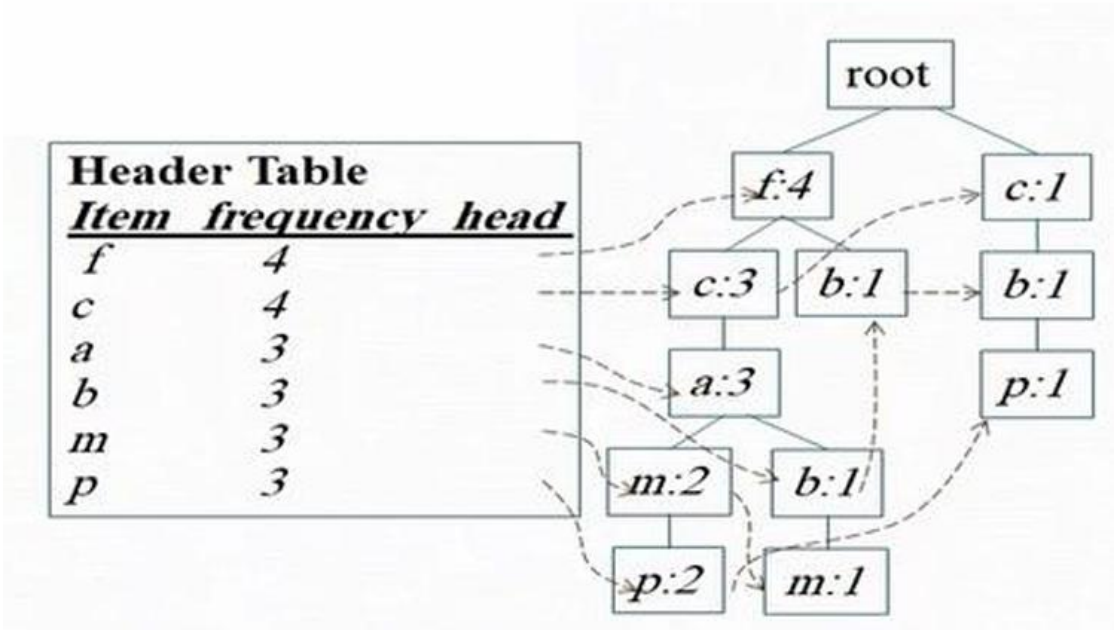
2- Order frequent items in descending order of their frequency

3- Scan DB again, construct FP-tree


FP- Tree construction
(ordered) frequent items
} {f, c, a, m, p}
{f, c, a, b, m}
{f, b}
{c, b, p}
} {f, c, a, m, p}
min_support = 3
Item frequency
f 4
c 4
a 3
b 3
m 3
p 3
root
f:1
c:1
a:1
m:1
p:1


(ordered) frequent items
} {f, c, a, m, p}
{f, c, a, b, m}
{f, b}
{c, b, p}
} {f, c, a, m, p}
min_support = 3
Item frequency
f 4
c 4
a 3
b 3
m 3
p 3
root
f:2
c:2
a:2
m:1
b:1
p:1
m:1
root
f:3
c:1
c:2
b:1
b:1
a:2
p:1
m:1
b:1
p:1
m:1

## FP- Tree construction




## Conditional Pattern-Base for the example

| Item | Conditional pattern-base | Conditional FP-tree |
|---|---|---|
| p | {(fcam:2), (cb:1)} | {(c:3)}\|p |
| m | {(fca:2), (fcab:1)} | {(f:3, c:3, a:3)}\|m |
| b | {(fca:1), (f:1), (c:1)} | Empty |
| a | {(fc:3)} | {(f:3, c:3)}\|a |
| c | {(f:3)} | {(f:3)}\|c |
| f | Empty | Empty |

## Interesting Measurements

- **Objective measures**

  Two popular measurements:

  **Support and Confidence**

- **Subjective measures**

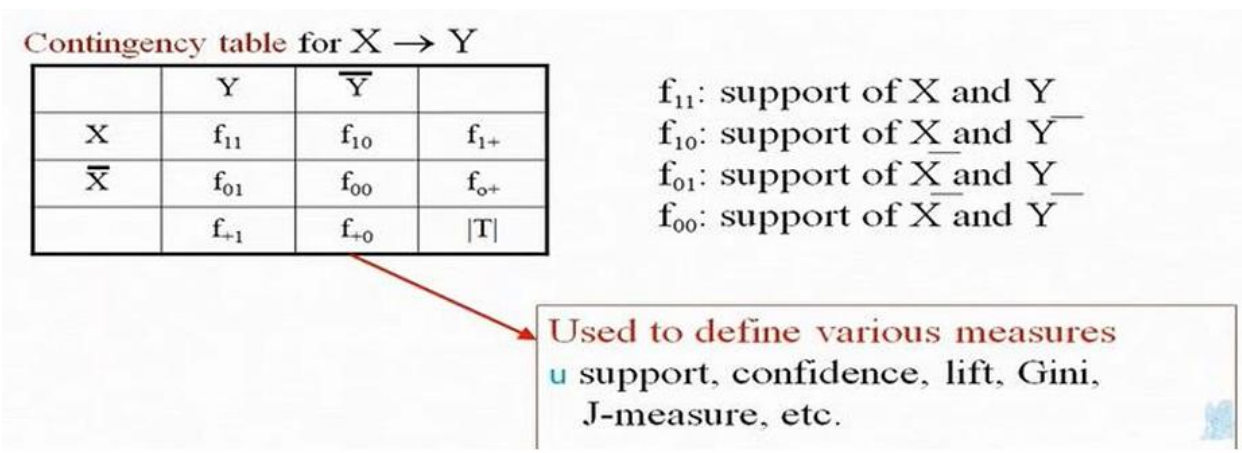
  A rule (pattern) is interesting if:

  It is unexpected (surprising to the user); and/or

  Actionable (the user can do something with it)

## Computing Interesting Measurements

Given a rule X→Y, information needed to compute rule interestingness can be obtained from a contingency table.



| | Coffee | $\overline{\text{Coffee}}$ | |
|---|---|---|---|
| Tea | 15 | 5 | 20 |
| $\overline{\text{Tea}}$ | 75 | 5 | 80 |
| | 90 | 10 | 100 |

Association Rule: Tea $\rightarrow$ Coffee

Confidence= P(Coffee|Tea) = 0.75

but P(Coffee) = 0.9

$\Rightarrow$ Although confidence is high, rule is misleading

$\Rightarrow$ P(Coffee|$\overline{\text{Tea}}$) = 0.9375

## Statistical –based measure

$$Interest = \frac{P(X,Y)}{P(X)P(Y)}$$

$$PS = P(X,Y) - P(X)P(Y)$$

$$\phi - coefficient = \frac{P(X,Y) - P(X)P(Y)}{\sqrt{P(X)[1-P(X)]P(Y)[1-P(Y)]}}$$

- $\phi$-coefficient is analogous to correlation coefficient for continuous variables

| | Y | $\overline{Y}$ | |
|---|---|---|---|
| X | 60 | 10 | 70 |
| $\overline{X}$ | 10 | 20 | 30 |
| | 70 | 30 | 100 |

$$\phi = \frac{0.6 - 0.7 \times 0.7}{\sqrt{0.7 \times 0.3 \times 0.7 \times 0.3}} = 0.5238$$

| | Y | $\overline{Y}$ | |
|---|---|---|---|
| X | 20 | 10 | 30 |
| $\overline{X}$ | 10 | 60 | 70 |
| | 30 | 70 | 100 |

$$\phi = \frac{0.2 - 0.3 \times 0.3}{\sqrt{0.7 \times 0.3 \times 0.7 \times 0.3}} = 0.5238$$

## **References**:

1. Han, J., Pei, J., Tong, H. (2022). Data mining: concepts and techniques. Morgan kaufmann.

2. Vaisman, A., Zimanyi, E. (2014). Data warehouse systems. Data-Centric Systems ´ and Applications, 9.